Vitis High-Level Synthesis User Guide (UG1399)

Interfaces of the HLS Design

Interfaces of the HLS Design

After the C++ code is complete, and synthesis is converting it to an RTL design, there are elements of the hardware implementation that are also in your control. Specifically, you need to consider the inputs to and the outputs from the HLS design, the layout of memory and managing data alignment, and the execution models of the HLS design. This section discusses the following topics:

- Defining Interfaces
- Vitis HLS Memory Layout Model
- Execution Modes of HLS Designs
- Controlling Initialization and Reset Behavior

Defining Interfaces

Introduction to Interface Synthesis

The arguments of the top-level function in an HLS component are synthesized into interfaces and ports that group multiple signals to define the communication protocol between the HLS component and elements external to the design. The tool defines interfaces automatically, using industry standards to specify the protocol used. The type of interfaces that the tool chooses depends on the data type and direction of the parameters of the top-level function, the target flow for the HLS component, the default interface configuration settings as described in Interface Configuration, and any specified INTERFACE pragmas or directives.

★ **Tip:** Interfaces can be manually assigned using the INTERFACE pragma or directive. Refer to Adding Pragmas and Directives for more information.

The target flows supported for an HLS component include the following as described in Target Flow Overview:

- The AMD Vivado™ IP flow which is the default flow for the tool
- The Vitis Kernel flow, which is the bottom-up design flow for the Vitis Application Acceleration Development flow

You can specify the target flow when creatingan HLS component as described in Vitis HLS Flow Steps.
The interface defines three elements of the kernel:

1. The interface defines channels for data to flow into or out of the HLS design. Data can flow from a variety of sources external to the kernel or IP, such as a host application, an external camera or sensor, or from another kernel or IP implemented on the AMD device. The default channels for Vitis kernels are AXI adapters as described in Interfaces for Vitis Kernel Flow.
2. The interface defines the port protocol that is used to control the flow of data through the data channel, defining when the data is valid and can be read or can be written, as defined in Port-Level Protocols for Vivado IP Flow.

   ★ **Tip:** These port protocols can be customized in the Vivado IP flow, but are set and cannot be changed in the Vitis kernel flow, in most cases.

   ---
3. The interface also defines the execution control scheme for the HLS design, specifying the operation of the kernel or IP as pipelined or sequential, as defined in Block-Level Control Protocols.

As described in Best Practices for Designing with M_AXI Interfaces the choice and configuration of interfaces is a key to the success of your design. However, the tool tries to simplify the process by selecting default interfaces for the target flows as described above.
After synthesis completes you can review the mapping of the software arguments of your C/C++ code to hardware ports or interfaces in the *SW I/O Information* section of the Synthesis Summary report.

Interfaces for Vitis Kernel Flow

The Vitis kernel flow provides support for compiled kernel objects (.xo) for software control from a host application and by the Xilinx Runtime (XRT). As described in PL Kernel Properties in the *Data Center Acceleration using Vitis* (UG1700), this flow has very specific interface requirements that Vitis HLS must meet.
Vitis HLS supports memory, stream, and register interface paradigms where each paradigm follows a certain interface protocol and uses the adapter to communicate with the external world.

- Memory Paradigm (`m_axi`): the data is accessed by the kernel through memory such as DDR, HBM, PLRAM/BRAM/URAM
- Stream Paradigm (`axis`): the data is streamed into the kernel from another streaming source, such as video processor or another kernel, and can also be streamed out of the kernel.
- Register Paradigm (`s_axilite`): The data is accessed by the kernel through register interfaces and accessed by software as register reads/writes.

★ **Tip:** The AXI protocol requires an active-Low reset. The tool defines this reset with a warning if the `syn.rtl.reset_level` is active-High, which is the default setting.

The Vitis kernel flow implements the following interfaces by default:

C-argument type	Paradigm	Interface protocol (I/O/Inout)
Scalar(pass by value)	Register	AXI4-Lite (`s_axilite`)
Array	Memory	AXI4 Memory Mapped (`m_axi`)
Pointer to array	Memory	`m_axi`
Pointer to scalar	Register	`s_axilite`
Reference	Register	`s_axilite`
hls::stream	Stream	AXI4-Stream (`axis`)

As you can see from the preceding table, a pointer to an array is implemented as an `m_axi` interface for data transfer, while a pointer to a scalar is implemented using the `s_axilite` interface. A scalar value passed as a constant does not need read access, while a pointer to a scalar value needs both read/write access. The `s_axilite` interface implements an additional internal protocol depending upon the C argument type. This internal implementation can be controlled using Port-Level Protocols for Vivado IP Flow. However, you should not modify the default port protocols in the Vitis kernel flow unless necessary.

✎ **Note:** Vitis HLS will not automatically infer the default interfaces for the member elements of a struct/class when the elements require different interface types. For example, when one element of a struct requires a stream interface while another member element requires an `s_axilite` interface. You must explicitly define an INTERFACE pragma for each element of the struct instead of relying on the default interface assignment. If no INTERFACE pragma or directive is defined Vitis HLS issues the following error message:

```
ERROR: [HLS 214-312] Vitis mode requires explicit INTERFACE
pragmas for structs in the interface. Please add one INTERFACE pragma for each struct
member field for argument 'd' of function 'dut(A&)' (example.cpp:19:0)
```

The default execution mode for Vitis kernel flow is pipelined execution, which enables overlapping execution of a kernel to improve throughput. This is specified by the `ap_ctrl_chain` block control protocol on the `s_axilite` interface.

★ **Tip:** The Vitis environment supports kernels with all of the supported block control protocols as described in Block-Level Control Protocols.

The `vadd` function in the following code provides an example of interface synthesis.

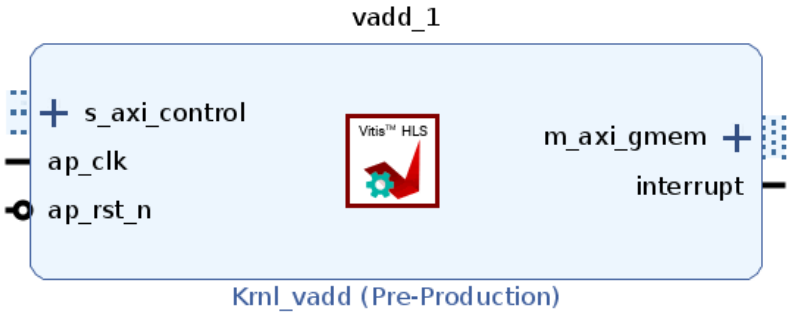
```
#define VDATA_SIZE 16

typedef struct v_datatype { unsigned int data[VDATA_SIZE]; } v_dt;

extern "C" {
void vadd(const v_dt* in1, // Read-Only Vector 1
          const v_dt* in2, // Read-Only Vector 2
          v_dt* out_r, // Output Result for Addition
          const unsigned int size // Size in integer
) {

   unsigned int vSize = ((size - 1) / VDATA_SIZE) + 1;

   // Auto-pipeline is going to apply pipeline to this loop
   vadd1:
   for (int i = 0; i < vSize; i++) {
      vadd2:
      for (int k = 0; k < VDATA_SIZE; k++) {
         out_r[i].data[k] = in1[i].data[k] + in2[i].data[k];
      }
   }
}
}
```

The vadd function includes:

- Two pointer inputs: `in1` and `in2`
- A pointer output: `out_r` that the results are written to
- A scalar value `size`

With the default interface synthesis settings used by Vitis HLS for the Vitis kernel flow, the design is synthesized into an RTL block with the ports and interfaces shown in the following figure.

Figure: RTL Ports After Default Interface Synthesis



The tool creates three types of interface ports on the RTL design to handle the flow of both data and control.

- Clock, Reset, and Interrupt ports: `ap_clk` and `ap_rst_n` and `interrupt` are added to the kernel.
- AXI4-Lite interface: `s_axi_control` interface which contains the scalar arguments like `size`, and manages address offsets for the m_axi interface, and defines the block control protocol.
- AXI4 memory mapped interface: `m_axi_gmem` interface which contains the pointer arguments: `in1`, `in2`, and `out_r`.

Details of M_AXI Interfaces for Vitis

AXI4 memory-mapped (`m_axi`) interfaces allow kernels to read and write data in global memory (DDR, HBM, PLRAM), Memory-mapped interfaces are a convenient way of sharing data across different elements of the accelerated application, such as between the host and kernel, or between kernels on the accelerator card. The main advantages for `m_axi` interfaces are listed below:

- The interface has independent read and write channels
- It supports burst-based accesses
- It provides a queue for outstanding transactions

Understanding Burst Access

AXI4 memory-mapped interfaces support high throughput bursts of up to 4K bytes with a single address phase. With burst mode transfers, Vitis HLS reads or writes data using a single base address followed by multiple sequential data samples, which makes this mode capable of higher data throughput. Burst mode of operation is possible when you use a pipelined `for` loop to copy memory. Refer to Controlling AXI4 Burst Behavior or AXI Burst Transfers for more information.

Automatic Port Widening and Port Width Alignment

As discussed in Automatic Port Width Resizing, Vitis HLS has the ability to automatically widen a port width to facilitate data transfers and improve burst access, if a burst access can be seen by the tool. Therefore all the preconditions needed for bursting, as described in AXI Burst Transfers, are also needed for port resizing.
In the Vitis Kernel flow automatic port width resizing is enabled by default with the following configuration commands (notice that one command is specified as bits and the other is specified as bytes):

```
syn.interface.m_axi_max_widen_bitwidth=512
syn.interface.m_axi_alignment_byte_size=64
```

Rules for Offset

!! Important: In the Vitis kernel flow the default mode of operation is *syn.interface.m_axi_offset=direct* and *syn.interface.default_slave_interface=s_axilite* and should not be changed.

The correct specification of the offset lets the HLS kernel correctly integrate into the Vitis system. Refer to Offset and Modes of Operation for more information.

Bundle Interfaces - Performance vs. Resource Utilization

By default, Vitis HLS groups function arguments with compatible options into a single `m_axi` interface adapter as described in M_AXI Bundles. Bundling ports into a single interface helps save device resources by eliminating AXI4 logic, which can be necessary when working in congested designs.
However, a single interface bundle can limit the performance of the kernel because all the memory transfers have to go through a single interface. The `m_axi` interface has independent READ and WRITE channels, so a single interface can read and write simultaneously, though only at one location. Using multiple bundles lets you increase the bandwidth and throughput of the kernel by creating multiple interfaces to connect to memory banks.

Details of S_AXILITE Interfaces for Vitis

In C++, a function starts to process data when the function is called from a parent function. The function call is pushed onto the stack when called, and removed from the stack when processing is complete to return control to the calling function. This process ensures the parent knows the status of the child.
Since the host and kernel occupy two separate compute spaces in the Vitis kernel flow, the "stack" is managed by the Xilinx Runtime (XRT), and communication is managed through the `s_axilite` interface. The kernel is software controlled through XRT by reading and writing the control registers of an `s_axilite` interface as described in S_AXILITE Control Register Map. The interface provides the following features:

Control Protocols

The block control protocol defines control registers in the `s_axilite` interface that let you set control signals to manage execution and operation of the kernel.

Scalar Arguments

Scalar inputs on a kernel are typical, and can be thought of as programming constants or parameters. The host application transfers these values through the `s_axilite` interface.

Pointers to Scalar Arguments

Vitis HLS lets you read to or write from a pointer to a scalar value when assigned to an `s_axilite` interface. Pointers are assigned by default to `m_axi` interfaces, so this requires you to manually assign the pointer to the `s_axilite` using the INTERFACE pragma or directive:

```
int top(int *a, int *b) {
#pragma HLS interface s_axilite port=a
```

Rules for Offset

Note: The Vitis kernel flow determines the required offsets. Do not specify the `offset` option in that flow.

Rules for Bundle

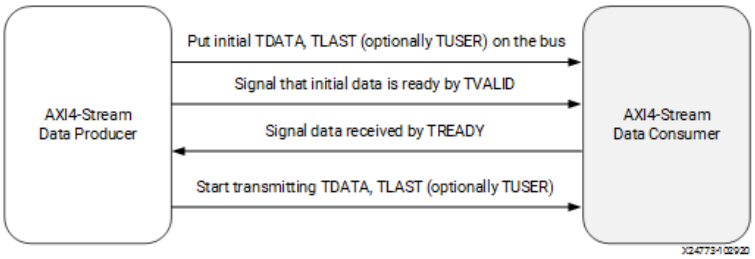
The Vitis kernel flow supports only a single `s_axilite` interface, which means that all `s_axilite` interfaces must be bundled together.

- When no bundle is specified the tool automatically creates a default bundle named `Control`.
- If for some reason you want to manually specify the bundle name, you must apply the same bundle to all `s_axilite` interfaces to create a single bundle.

Details of AXIS Interfaces for Vitis

The AXI4-Stream protocol (AXIS) defines a single uni-directional channel for streaming data in a sequential manner. The AXI4-Stream interfaces can burst an unlimited amount of data, which significantly improves performance. Unlike the AXI4 memory-mapped interface which needs an address to read/write the memory, the AXIS interface simply passes data to another AXIS interface without needing an address, and so uses fewer device resources. Combined, these features make the streaming interface a light-weight high performance interface.
The AXI4-Stream works on an industry-standard `ready`/`valid` handshake between a producer and consumer, as shown in the figure below. The data transfer is started after the producer sends the `TVALID` signal, and the consumer responds by sending the `TREADY` signal. This handshake of data and control should continue until either `TREADY` or `TVALID` are set low, or the producer asserts the `TLAST` signal indicating it is the last data packet of the transfer.

Figure: AXI4-Stream Handshake



Important: The AXIS interface can only be assigned to the top-level arguments (ports) of a kernel or IP, and cannot be assigned to the arguments of functions internal to the design. Streaming channels used inside the HLS design should use `hls::stream` and not an AXIS interface.

You should define the streaming data type using `hls::stream<T_data_type>`, and use the `ap_axis` struct type to implement the AXIS interface. As explained in AXI4-Stream Interfaces the `ap_axis` struct lets you choose the implementation of the interface as with or without side-channels:

- AXI4-Stream Interfaces without Side-Channels implements the AXIS interface as a very light-weight interface using fewer resources
- AXI4-Stream Interfaces with Side-Channels implements a full featured interface providing greater control

★ **Tip:** You should not define your own struct for modeling the AXIS signals (side channels, TLAST, TVALID). Instead you can overload the TDATA signal for implementing your data type.

Interfaces for Vivado IP Flow

The Vivado IP flow supports a wide variety of I/O protocols and handshakes due to the requirement of supporting FPGA design for a wide variety of applications. This flow supports a traditional system design flow where multiple IP are integrated into a system. IP can be generated through Vitis HLS. In this IP flow there are two modes of control for execution of the system:

- Software Control: The system is controlled through a software application running on an embedded Arm processor or external x86 processor, using drivers to access elements of the hardware design, and reading and writing registers in the hardware to control the execution of IP in the system.
- Self Synchronous: In this mode the IP exposes signals which are used for starting and stopping the kernel. These signals are driven by other IP or other elements of the system design that handles the execution of the IP.

The Vivado IP flow supports memory, stream, and register interface paradigms where each paradigm supports different interface protocols to communicate with the external world, as shown in the following table. Note that while the Vitis kernel flow supports only the AXI4 interface adapters, this flow supports a number of different interface types.

Table: Interface Types

Paradigm	Description	Interface Types
Memory	Data is accessed by the kernel through memory such as DDR, HBM, PLRAM/BRAM/URAM	`ap_memory`, AXI4 Memory Mapped (`m_axi`)
Stream	Data is streamed into the kernel from another streaming source, such as video processor or another kernel, and can also be streamed out of the kernel.	`ap_fifo`, AXI4-Stream (`axis`)
Register	Data is accessed by the kernel through register interfaces performed by register reads and writes.	`ap_none`, `ap_hs`, `ap_ack`, `ap_ovld`, `ap_vld`, and AXI4-Lite adapter (`s_axilite`).

The default interfaces are defined by the C-argument type in the top-level function, and the default paradigm, as shown in the following table.

Table: Default Interfaces

C-Argument Type	Supported Paradigms	Default Paradigm	Default Interface Protocol		
			Input	Output	Inout
Scalar variable (pass by value)	Register	Register	ap_none	N/A	N/A
Array	Memory, Stream	Memory	ap_memory	ap_memory	ap_memory
Pointer	Memory, Stream, Register	Register	ap_none	ap_vld	ap_ovld

C-Argument Type	Supported Paradigm	Default Paradigm	Default Interface Protocol		
			Input	Output	Inout
Reference	Register	Register	ap_none	ap_vld	ap_vld
`hls::stream`	Stream	Stream	ap_fifo	ap_fifo	N/A

The default execution mode for Vivado IP flow is sequential execution, which requires the HLS IP to complete one iteration before starting the next. This is specified by the `ap_ctrl_hs` block control protocol. The control protocol can be changed as specified in Block-Level Control Protocols.
The `vadd` function in the following code provides an example of interface synthesis in the Vivado IP flow.

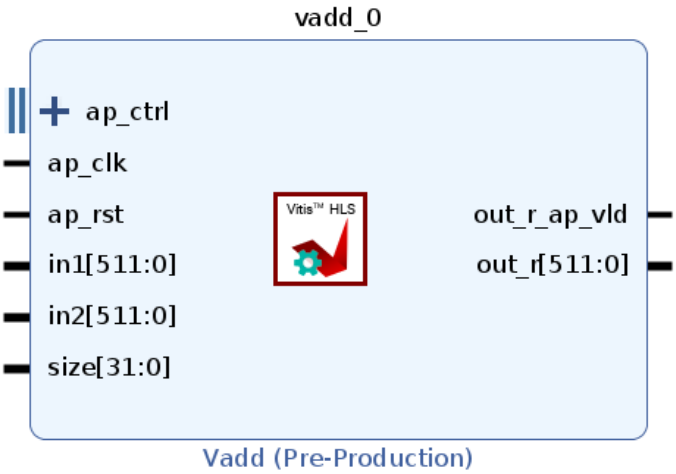
```
#define VDATA_SIZE 16

typedef struct v_datatype { unsigned int data[VDATA_SIZE]; } v_dt;

extern "C" {
void vadd(const v_dt* in1, // Read-Only Vector 1
          const v_dt* in2, // Read-Only Vector 2
          v_dt* out_r, // Output Result for Addition
          const unsigned int size // Size in integer
) {

   unsigned int vSize = ((size - 1) / VDATA_SIZE) + 1;

   // Auto-pipeline is going to apply pipeline to this loop
   vadd1:
   for (int i = 0; i < vSize; i++) {
      vadd2:
      for (int k = 0; k < VDATA_SIZE; k++) {
         out_r[i].data[k] = in1[i].data[k] + in2[i].data[k];
      }
   }
}
}
```

The `vadd` function includes:

- Two pointer inputs: `in1` and `in2`
- A pointer: `out_r` that the results are written to
- A scalar value `size`

With the default interface synthesis settings used for the Vivado IP flow, the design is synthesized into an RTL block with the ports and interfaces shown in the following figure.

Figure: RTL Ports After Default Interface Synthesis



In the default Vivado IP flow the tool creates three types of interface ports on the RTL design to handle the flow of both data and control.

- Clock and Reset ports: `ap_clk` and `ap_rst` are added to the kernel.
- Block-level control protocol: The `ap_ctrl` interface is implemented as an `s_axilite` interface.
- Port-level interface protocols: These are created for each argument in the top-level function and the function return (if the function returns a value). As explained in the table above most of the arguments use a port protocol of `ap_none`, and so have no control signals. In the `vadd` example above these ports include: `in1`, `in2`, and `size`. However, the `out_r_o` output port uses the `ap_vld` protocol and so is associated with the `out_r_o_ap_vld` signal.

AP_Memory in the Vivado IP Flow

The `ap_memory` is the default interface for the memory paradigm described in the tables above. In the Vivado IP flow it is used for communicating with memory resources such as BRAM and URAM. The `ap_memory` protocol also follows the address and data phase. The protocol initially requests to read/write the resource and waits until it receives an acknowledgment of the resource availability. It then initiates the data transfer phase of read/write.
An important consideration for `ap_memory` is that it can only perform a single beat data transfer to a single address, which is different from `m_axi` which can do burst accesses. This makes the `ap_memory` a lightweight protocol, compared to the others.

- Memory Resources: By default Vitis HLS implements a protocol to communicate with a single-port RAM resource. You can control the implementation of the protocol by specifying the `storage_type` as part of the INTERFACE pragma or directive. The `storage_type` lets you explicitly define which type of RAM is used, and which RAM ports are created (single-port or dual-port). If no `storage_type` is specified Vitis HLS uses:
  - A single-port RAM by default.
  - A dual-port RAM if it reduces the initiation interval or latency.

M_AXI Interfaces in the Vivado IP Flow

AXI4 memory-mapped (`m_axi`) interfaces allow an IP to read and write data in global memory (DDR, HBM, PLRAM), Memory-mapped interfaces are a convenient way of sharing data across multiple IP. The main advantages for `m_axi` interfaces are listed below:

- The interface has independent read and write channels
- It supports burst-based accesses
- It provides a queue for outstanding transactions

Understanding Burst Access

AXI4 memory-mapped interfaces support high throughput bursts of up to 4K bytes with just a single address phase. With burst mode transfers, Vitis HLS reads or writes data using a single base address followed by multiple sequential data samples, which makes this mode capable of higher data throughput. Burst mode of operation is possible when you use the C `memcpy` function or a pipelined `for` loop. Refer to Controlling AXI4 Burst Behavior or AXI Burst Transfers for more information.

Automatic Port Widening and Port Width Alignment

As discussed in Automatic Port Width Resizing, Vitis HLS has the ability to automatically widen a port width to facilitate data transfers and improve burst access when all the preconditions needed for bursting are present. In the Vivado IP flow the following configuration settings disable automatic port width resizing by default. To enable this feature you must change these configuration options (notice that one command is specified as bits and the other is specified as bytes):

```
syn.interface.m_axi_max_widen_bitwidth=0
syn.interface.m_axi_alignment_byte_size=0
```

Specifying Alignment for Vivado IP mode

The alignment for an `m_axi` port allows the port to read and write memory according to the specified alignment. Choosing the correct alignment is important as it will impact performance in the best case, and can impact functionality in the worst case.

Aligned memory access means that the pointer (or the start address of the data) is a multiple of a type-specific value called the alignment. The alignment is the natural address multiple where the type must be or should be stored (for example for performance reasons) on a Memory. For example, standard 32-bit architecture stores words of 32 bits, each of 4 bytes in the memory. The data is aligned to one-word or 4-byte boundary.

The alignment should be consistent in the system. The alignment is determined when the IP is operating in AXI4 master mode and should be specified, like the 32-bit architecture with 4-byte alignment. When the IP is operating in slave mode the alignment should match the alignment of the master.

Rules for Offset

The default for `m_axi` offset is *offset=direct* and *syn.interface.default_slave_interface=s_axilite*. However, in the Vivado IP flow you can change it as described in Offset and Modes of Operation.

Bundle Interfaces - Performance vs. Resource Utilization

By default, Vitis HLS groups function arguments with compatible options into a single `m_axi` interface adapter as described in M_AXI Bundles. Bundling ports into a single interface helps save device resources by eliminating AXI4 logic, which can be necessary when working in congested designs.

However, a single interface bundle can limit the performance of the IP because all the memory transfers have to go through a single interface. The `m_axi` interface has independent READ and WRITE channels, so a single interface can read and write simultaneously, though only at one location. Using multiple bundles lets you increase performance by creating multiple interfaces to connect to memory banks.

S_AXILITE in the Vivado IP Flow

In the Vivado IP flow, the default execution control is managed by register reads and writes through an `s_axilite` interface using the default `ap_ctrl_hs` control protocol. The IP is software controlled by reading and writing the control registers of an `s_axilite` interface as described in S_AXILITE Control Register Map.

The `s_axilite` interface provides the following features:

Control Protocols

The block control protocol as specified in Block-Level Control Protocols.

Scalar Arguments

Scalar arguments from the top-level function can be mapped to an `s_axilite` interface which creates a register for the value as described in S_AXILITE Control Register Map. The software can perform reads/writes to this register space.

Rules for Offset

The Vivado IP flow defines the size, or range of addresses assigned to a port based on the data type of the associated C-argument in the top-level function. However, the tool also lets you manually define the offset size as described in S_AXILITE Offset Option.

Rules for Bundle

In the Vivado IP flow you can specify multiple bundles using the `s_axilite` interface, and this will create a separate interface adapter for each bundle you have defined. However, there are some rules related to using multiple bundles that you should be familiar with as explained in S_AXILITE Bundle Rules.

AP_FIFO in the Vivado IP Flow

In the Vivado IP flow, the `ap_fifo` interface protocol is the default interface for the streaming paradigm on the interface for communication with a memory resource FIFO, and can also be used as a communication channel between different functions inside the IP. This protocol should only be used if the data is accessed sequentially, and AMD *strongly recommends* using the `hls::stream<data type>` which implements a FIFO.

★ **Tip:** The `<data type>` should not be the same as the `T_data_type`, which should only be used on the interface.

AXIS Interfaces in the Vivado IP Flow

The AXI4-Stream protocol (`axis`) is an alternative for streaming interfaces, and defines a single uni-directional channel for streaming data in a sequential manner. Unlike the `m_axi` protocol, the AXI4-Stream interfaces can burst an unlimited amount of data, which significantly improves performance. Unlike the AXI4 memory-mapped interface which needs an address to read/write the memory, the `axis` interface simply passes data to another `axis` interface without needing an address, and so uses fewer device resources. Combined, these features make the streaming interface a light-weight high performance interface as described in AXI4-Stream Interfaces.

AXI Adapter Interface Protocols

!! **Important:** As discussed in Interfaces for Vitis Kernel Flow, the AXI4 adapter interfaces are the default interfaces used by Vitis HLS for the Vitis Application Acceleration Development flow, though they are also supported in the Vivado IP flow. The AXI4-Stream Accelerator Adapter is a soft AMD LogiCORE™ Intellectual Property (IP) core used as a infrastructure block for connecting hardware accelerators to embedded CPUs.

The AXI4 interfaces supported by Vitis HLS include the AXI4-Stream interface (`axis`), AXI4-Lite (`s_axilite`), and AXI4 master (`m_axi`) interfaces. For a complete description of the AXI4 interfaces, including timing and ports, see the *Vivado Design Suite: AXI Reference Guide* (UG1037). As described in the following sections, the AXI4 interfaces implement an adapter to manage communication according to the protocol. None of the other available Vitis HLS interfaces implement such an adapter.

m_axi

Specify on arrays and pointers (and references in C++) only. The `m_axi` mode specifies an AXI4 Memory Mapped interface.

★ **Tip:** You can group bundle arguments into a single `m_axi` interface.

s_axilite

Specify this protocol on any type of argument except streams. The `s_axilite` mode specifies an AXI4-Lite slave interface.

★ **Tip:** You can bundle multiple arguments into a single `s_axilite` interface.

axis

Specify this protocol on input arguments or output arguments only, not on input/output arguments. The `axis` mode specifies an AXI4-Stream interface.

★ **Tip:** The AXI protocol requires an active-Low reset. If your design uses AXI interfaces the tool will define this reset level with a warning if the `syn.rtl.reset_level` is active-High, which is the default setting.

AXI4 Master Interface

AXI4 memory-mapped (`m_axi`) interfaces allow kernels to read and write data in global memory (DDR, HBM, PLRAM). Memory-mapped interfaces are a convenient way of sharing data across different elements of the accelerated application, such as between the host and kernel, or between kernels on the accelerator card. Refer to Vitis-HLS-Introductory-Examples/Interface/Memory on GitHub for examples of some of these concepts.
The main advantages for `m_axi` interfaces are listed below:

- The interface has a separate and independent read and write channels
- It supports burst-based accesses with potential performance of ~17 GBps
- It provides support for outstanding transactions
- It can include cache to improve performance as indicated by the CACHE pragma or directive.

In the Vitis Kernel flow the `m_axi` interface is assigned by default to pointer and array arguments. In this flow it supports the following default features:

- Pointer and array arguments are automatically mapped to the `m_axi` interface
- The default mode of operation is `offset=slave` in the Vitis flow and should not be changed
- All pointer and array arguments are mapped to a single interface bundle to conserve device resources, and ports share read and write access across the time it is active
- The default alignment in the Vitis flow is set to 64 bytes
- The maximum read/write burst length is set to 16 by default

While not used by default in the Vivado IP flow, when the `m_axi` interface is specified it has the following default features:

- The default operation mode is *offset=off* but you can change it as described in Offset and Modes of Operation
- Assigned pointer and array arguments are mapped to a single interface bundle to conserve device resources, and share the interface across the time it is active
- The default alignment in Vivado IP flow is set to 1 byte
- The maximum read/write burst length is set to 16 by default

In both the Vivado IP flow and Vitis kernel flow, global default values for the `m_axi` interface are defined by the `syn.interface.xxx` commands as described in Interface Configuration. The INTERFACE pragma or directive can be used to modify default values as needed for specific interfaces. Some customization can help improve design performance as described in Optimizing AXI System Performance. You can also specify the CACHE pragma or directive as described in pragma HLS cache to improve performance of the `m_axi` interface.
You can use an AXI4 master interface on array or pointer/reference arguments, which Vitis HLS implements in one of the following modes:

- Individual data transfers
- Burst mode data transfers

With individual data transfers, Vitis HLS reads or writes a single element of data for each address. The following example shows a single read and single write operation. In this example, Vitis HLS generates an address on the AXI interface to

read a single data value and an address to write a single data value. The interface transfers one data value per address.

```
void bus (int *d) {
 static int acc = 0;

 acc += *d;
 *d  = acc;
}
```

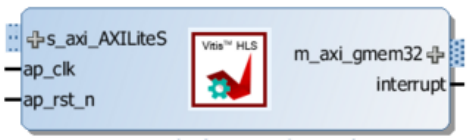
With burst mode transfers, Vitis HLS reads or writes data using a single base address followed by multiple sequential data samples, which makes this mode capable of higher data throughput. Burst mode of operation is possible when using a pipelined `for` loop. Refer to AXI Burst Transfers for more information.

!! Important: The use of `memcpy` is discouraged because it can not be inlined or pipelined. Its use can also be problematic because it changes the type of the argument into `char`, which can lead to errors if `array_partition`, `array_reshape`, or struct `disaggregate` is used. Instead you are recommended to write your own version of `memcpy` with explicit arrays and loops to provide better control.

When the prior example is synthesized, it results in the interfaces shown in the following figure.

Note: In this figure, the AXI4 interfaces are collapsed.

Figure: AXI4 Interface



When using a `for` loop to implement burst reads or writes, follow these requirements:

- Pipeline the loop
- Access addresses in increasing order
- Do not place accesses inside a conditional statement
- For nested loops, do not manually flatten loops (using LOOP_FLATTEN pragma or directive), because this inhibits the burst operation

Note: Only one read and one write is allowed in a `for` loop unless the ports are bundled in different AXI ports.

Offset and Modes of Operation

!! Important: In the Vitis kernel flow the default mode of operation is *offset=slave* and should not be changed. In the Vivado IP flow the default is also *offset=slave*, but can be changed.

The AXI4 Master interface has a read/write address channel that can be used to read/write specific addresses. By default the `m_axi` interface starts all read and write operations from the address `0x00000000`. For example, given the following code, the design reads data from addresses `0x00000000` to `0x000000C7` (50 32-bit words, gives 200 bytes), which represents 50 address values. The design then writes data back to the same addresses.

```
#include <stdio.h>
#include <string.h>

void example(int *a){

#pragma HLS INTERFACE mode=m_axi port=a depth=50

  int i;
  int buff[50];

  //The use of memcpy is discouraged because multiple calls of memcpy cannot be
```

```
  //pipelined and will be scheduled sequentially. Use a dedicated for loop instead
  //memcpy(buff,(const int*)a,50*sizeof(int));
  //Burst read input
  for(i=0; i<50; i++){
    buff[i] = a[i];
  }

  // Compute value
  for(i=0; i < 50; i++){
    buff[i] = buff[i] + 100;
  }

  //Burst write output
  for(i=0; i<50; i++){
    a[i] = buff[i];
  }
}
```

The HLS compiler provides the capability to let the base address be configured statically in the Vivado IP for instance, or dynamically by the software application or another IP during runtime.
The `m_axi` interface can be both a master initiating transactions, and also a slave interface that receives the data and sends acknowledgment. Depending on the mode specified with the `offset` option of the INTERFACE pragma, an HLS IP can use multiple approaches to set the base address.

★ **Tip:** The `syn.interface.m_axi_offset` configuration command provides a global setting for the offset, that can be overridden for specific `m_axi` interfaces using the INTERFACE pragma `offset` option.

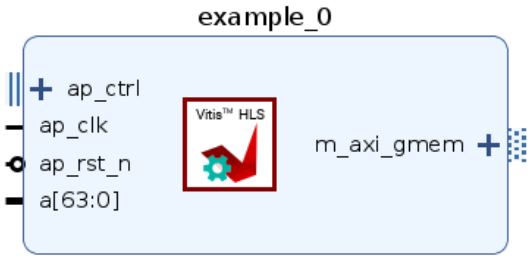
- *Master Mode*: When acting as a master interface with different `offset` options, the `m_axi` interface start address can be either hard-coded or set at runtime.
  - `offset=off`: Sets the base address of the `m_axi` interface to 0x00000000 and it cannot be changed in the Vivado IP integrator. One disadvantage with this approach is that you cannot change the base address during runtime. See Customizing AXI4 Master Interfaces in IP Integrator for setting the base address.
    The following shows the INTERFACE pragma setting `offset=off` for the function.

    ```
    void example(int *a){
    #pragma HLS INTERFACE m_axi depth=50 port=a offset=off
    ...
    }
    ```

  - `offset=direct`: Vitis HLS generates a port on the IP for setting the address. Note the addition of the a port as shown in the figure below. This lets you update the address at runtime, so you can have one `m_axi` interface reading and writing different locations. For example, an HLS module that reads data from an ADC into RAM, and an HLS module that processes that data. Because you can change the address on the module, while one HLS module is processing the initial dataset the other module can be reading more data into different address.

    ```
    void example(int *a){
    #pragma HLS INTERFACE m_axi depth=50 port=a offset=direct
    ...
    }
    ```

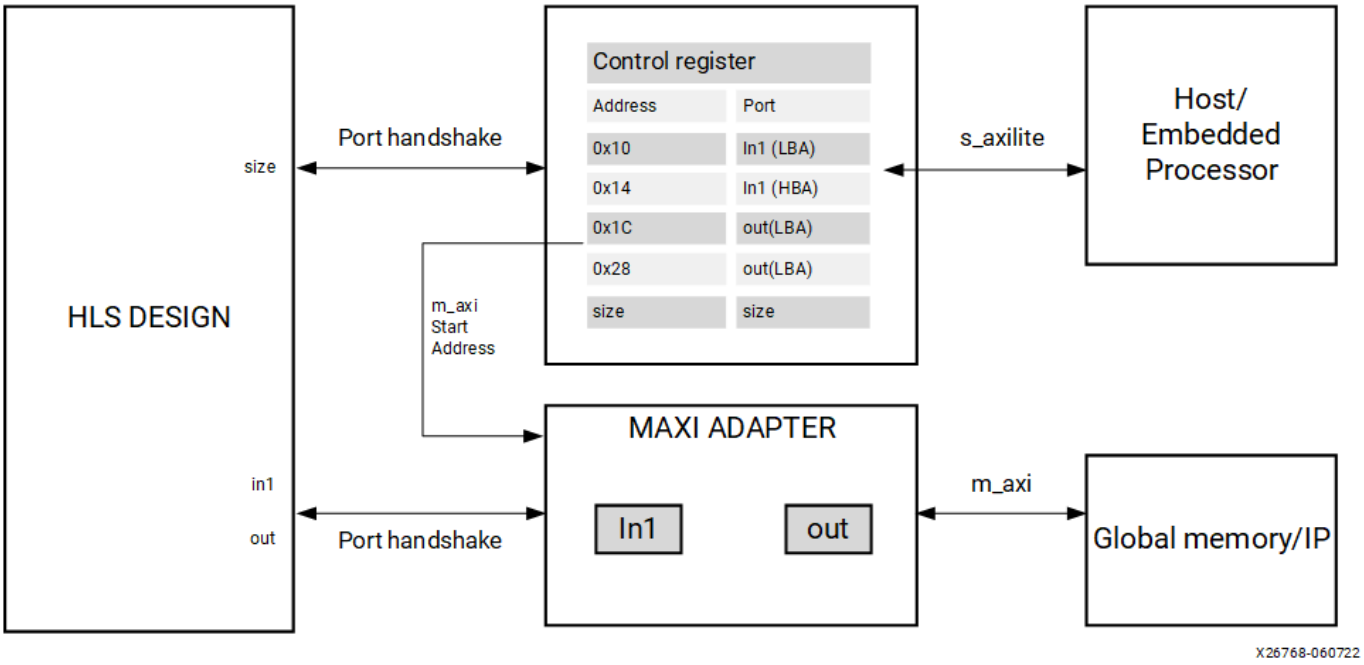
Figure: offset=direct



- *Slave Mode*: The slave mode for an interface is set with `offset=slave`. In this mode the IP will be controlled by the host application, or the micro-controller through the `s_axilite` interface. This is the default for both the Vitis kernel flow, and the Vivado IP flow. Here is the flow of operation:
  1. initially, the Host/CPU will start the IP or kernel using the block-level control protocol which is mapped to the `s_axilite` adapter.
  2. The host will send address offsets for the `m_axi` interfaces through the `s_axilite` adapter.
  3. The `m_axi` adapter will read the start address from the `s_axilite` adapter and store it in a queue.
  4. The HLS design starts to read the data from the global memory.

As shown in the following figure, the HLS design will have both the `s_axilite` adapter for the base address, and the `m_axi` to perform read and write transfer to the global memory.

Figure: AXI Adapters in Slave Mode



Offset Rules

The following are rules associated with the `offset` option:

- Fully Specified Offset: When the user explicitly sets the offset value the tool uses the specified settings. The user can also set different offset values for different `m_axi` interfaces in the design, and the tool will use the specified offsets.

```
#pragma HLS INTERFACE s_axilite port=return
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=out offset=direct
#pragma HLS INTERFACE mode=m_axi bundle=BUS_B port=in1 offset=slave
#pragma HLS INTERFACE mode=m_axi bundle=BUS_C port=in2 offset=off
```

- No Offset Specified: If there are no offsets specified in the INTERFACE pragma, the tool will defer to the setting specified by `syn.interface.m_axi_offset`.

Note: If the global `syn.interface.m_axi_offset` setting is specified, and the design has an `s_axilite` interface, the global setting is ignored and `offset=slave` is assumed.

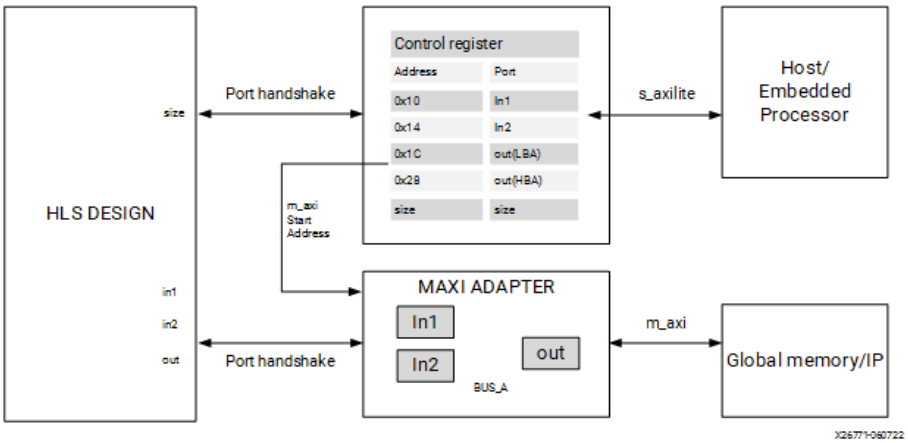
```
void top(int *a) {
#pragma HLS interface mode=m_axi port=a
#pragma HLS interface mode=s_axilite port=a
}
```

M_AXI Bundles

The Vitis Unified IDE and v++ command groups function arguments with compatible options into a single `m_axi` interface adapter. Bundling ports into a single interface helps save FPGA resources by eliminating AXI logic, but it can limit the performance of the kernel because all the memory transfers have to go through a single interface. The `m_axi` interface has independent READ and WRITE channels, so a single interface can read and write simultaneously, though only at one location. Using multiple bundles the bandwidth and throughput of the kernel can be increased by creating multiple interfaces to connect to multiple memory banks.
In the following example all the pointer arguments are grouped into a single `m_axi` adapter using the interface option `bundle=BUS_A`, and adds a single `s_axilite` adapter for the `m_axi` offsets, the scalar argument `size`, and the function return.

```
extern "C" {
void vadd(const unsigned int *in1, // Read-Only Vector 1
          const unsigned int *in2, // Read-Only Vector 2
          unsigned int *out,     // Output Result
          int size                 // Size in integer
          ) {

#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=out
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=in1
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=in2
#pragma HLS INTERFACE mode=s_axilite port=in1
#pragma HLS INTERFACE mode=s_axilite port=in2
#pragma HLS INTERFACE mode=s_axilite port=out
#pragma HLS INTERFACE mode=s_axilite port=size
#pragma HLS INTERFACE mode=s_axilite port=return
```

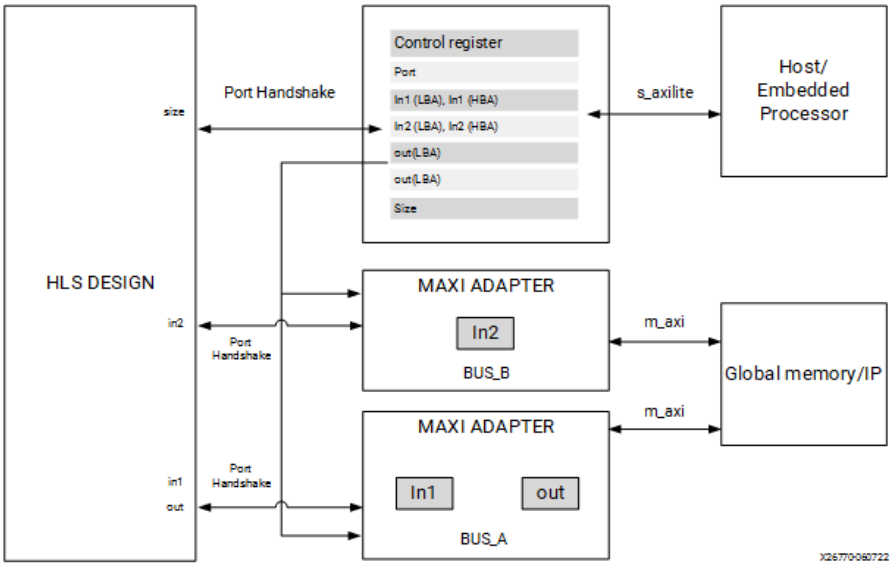
Figure: MAXI and S_AXILITE



You can also choose to bundle function arguments into separate interface adapters as shown in the following code. Here the argument `in2` is grouped into a separate interface adapter with `bundle=BUS_B`. This creates a new `m_axi` interface adapter for port `in2`.

```
extern "C" {
void vadd(const unsigned int *in1, // Read-Only Vector 1
          const unsigned int *in2, // Read-Only Vector 2
          unsigned int *out,     // Output Result
          int size                 // Size in integer
          ) {

#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=out
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=in1
#pragma HLS INTERFACE mode=m_axi bundle=BUS_B port=in2
#pragma HLS INTERFACE mode=s_axilite port=in1
#pragma HLS INTERFACE mode=s_axilite port=in2
#pragma HLS INTERFACE mode=s_axilite port=out
#pragma HLS INTERFACE mode=s_axilite port=size
#pragma HLS INTERFACE mode=s_axilite port=return
```

Figure: 2 MAXI Bundles



Bundle Rules

The global configuration command `syn.interface.m_axi_auto_max_ports=false` will limit the number of interface bundles to the minimum required. It will allow the tool to group compatible ports into a single `m_axi` interface. The default setting for this command is disabled (false), but you can enable it to maximize bandwidth by creating a separate `m_axi` adapter for each port.
With `m_axi_auto_max_ports` disabled, the following are some rules for how the tool handles bundles under different circumstances:

1. **Default Bundle Name**: The tool groups all interface ports with no bundle name into a single `m_axi` interface port using the tool default name `bundle=<default>`, and names the RTL port `m_axi_<default>`. The following pragmas:

   ```
   #pragma HLS INTERFACE mode=m_axi port=a depth=50
   #pragma HLS INTERFACE mode=m_axi port=a depth=50
   #pragma HLS INTERFACE mode=m_axi port=a depth=50
   ```

   Result in the following messages:

   ```
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
   ```

2. User-Specified Bundle Names: The tool groups all interface ports with the same user-specified `bundle=<string>` into the same `m_axi` interface port, and names the RTL port the value specified by `m_axi_<string>`. Ports without `bundle` assignments are grouped into the default bundle as described above. The following pragmas:

   ```
   #pragma HLS INTERFACE mode=m_axi port=a depth=50 bundle=BUS_A
   #pragma HLS INTERFACE mode=m_axi port=b depth=50
   #pragma HLS INTERFACE mode=m_axi port=c depth=50
   ```

   Result in the following messages:

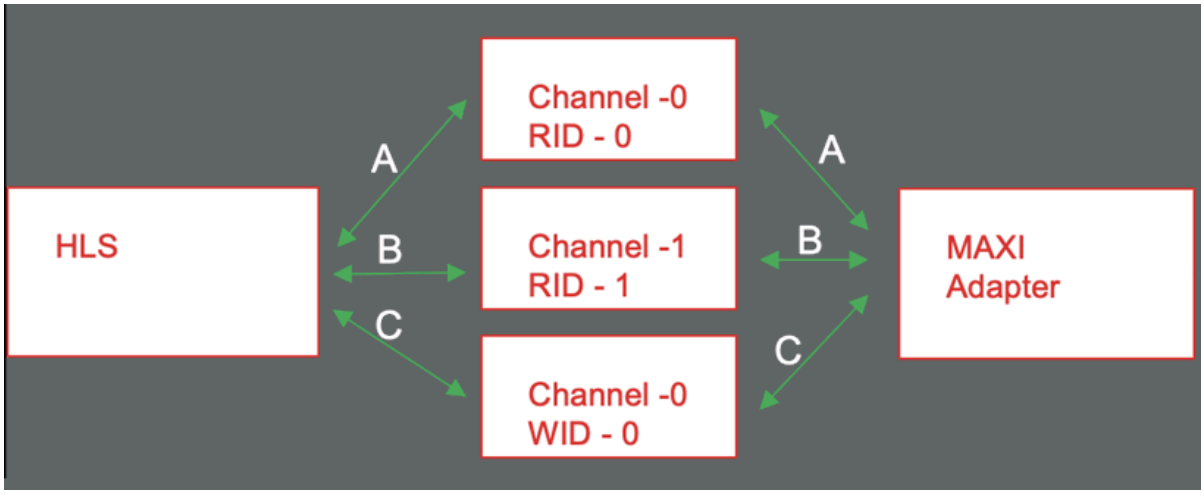
   ```
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/BUS_A' to 'm_axi'.
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to 'm_axi'.
   ```

!! Important: If you bundle incompatible interfaces the tool issues a message and ignores the bundle assignment.

M_AXI Channels

M_AXI channels implement a separate channel for each pointer argument mapped to a single AXI interface, rather than requiring a separate adapter.

Figure: Maxi Adapter



This enables the following benefits:

- The kernel uses fewer M_AXI adapters and consumes fewer hardware resources.
- Multiple pointer arguments mapped to a single AXI interface can be used inside a dataflow region.
- Using a unique AXI ID for each pointer argument enables burst interleaving which can result in higher utilization of the AXI bus bandwidth.

There are two methods to enable this feature in your design:

- Enable globally on `m_axi` interfaces using the `syn.interface.m_axi_auto_id_channel=true` configuration command as described in Interface Configuration. The HLS tool automatically adds channels to the `m_axi` adapter when this is enabled.
- Enable on a specific `m_axi` interface using the `channel` option of the INTERFACE pragma or directive as described in syn.directive.interface.

From the architectural viewpoint, each MAXI bundle is split into two parts:

- One load and/or store unit (LSU) per channel and per direction (For example, if an input pointer and an output pointer have the same channel ID, you get one load unit and one store unit).
  - If you do not use the channel option, there is just one load unit and one store unit for all the MAXI top pointers mapped to that bundle.
  - The LSU contains the cache, if enabled. The picture refers to LSUs with the cache. If there is no cache, then there is no tag mem and no valid mem, of course.
  - If multiple top pointers are mapped to the same channel (or to the default single channel), they must all have the same cache config, or no cache.
  - The LSU contains the data and address buffers that enable as many kernel-side bursts to be executed as required by the burst length and number of pending requests configured by the user or auto-inferred by the FE. It is the most BRAM-intensive part of the adapter.
  - Each LSU has its own AXI ID, hence different LSUs could generate interleaved bursts, hence more channels imply better overall throughput if multiple top pointers mapped to the same bundle are accessed concurrently by the kernel.
- One read adapter and one write adapter.
  - The adapters do not buffer the data but buffer the pending requests until the responses return.
  - They break the potentially very long bursts generated by the kernel into chunks that satisfy the AXI-mandated burst length restrictions.

M_AXI Resources

The AXI Master Adapter converts the customized AXI commands from the HLS scheduler to standard AXI AMBA® protocol and sends them to the external memory. The MAXI adapter uses resources such as FIFO to store the requests/Data and ack. Here is the summary of the modules and the resource they consume:

- Write Module: The bus write modules performs the write operations.
  - `FIFO_wreq`: This FIFO module stores the future write requests. When the AW channel is available a new write request to global memory will be popped out of this FIFO.
  - `buff_wdata`: This FIFO stores the future write data that needs to be sent to the global memory. When the W channel is available and AXI protocol conditions are met, the write data of size= burst_length will be popped out of this FIFO and sent to the global memory.
  - `FIFO_resp`: This module is responsible for controlling the number of pipelined write responses sent to the module.
- Read Module: These modules perform the read operations. It uses the following resources
  - `FIFO_rreq`: This FIFO module stores the future write requests. When the AR channel is free a read request to global memory will be popped out of this FIFO.
  - `buff_rdata`: This FIFO stores the read data that are received from the global memory.

The device resource consumption of the M_AXI adapter is a sum of all the write modules (size of the `FIFO_wreq module`, `buff_wdata`, and size of FIFO_ resp) and the sum of all read modules. In general, the size of the FIFO is calculated as = Width * Depth. When you refer to a 1KB FIFO storage it can be one of the configurations such as 32*32, 8*64 etc, which are selected according to the design specification. Similarly, the adapter FIFO storage can be globally configured for the design using the following configuration commands:

- `syn.interface.m_axi_latency`
- `syn.interface.m_axi_max_read_burst_length/syn.interface.m_axi_max_write_burst_length`
- `syn.interface.m_axi_num_read_outstanding/syn.interface.m_axi_num_write_outstanding`
- `syn.interface.m_axi_addr64`

★ **Tip:** You can also use similar options on the INTERFACE pragma or directive to configure specific m_axi interfaces.

These configuration options control the width and depth of the FIFO as shown below.

- Size of the `FIFO_wreq/rreq module` = (width(`syn.interface.m_axi_addr64`)) * Depth(`syn.interface.m_axi_latency`)). This FIFO will be implemented as a shift register by the Vivado tool.
- Size of the `buff_wdata/buff_rdata` module = (width (port width after HLS synthesis) * Depth (`syn.interface.m_axi_num_write_outstanding` * `syn.interface.m_axi_max_write_burst_length`)).

  ★ **Tip:** This FIFO by default will be implemented as block RAM, but it can be implemented in LUTRAM or URAM as determined by `syn.interface.m_axi_buffer_impl`.
- Size of the `FIFO_resp` module = depth (`syn.interface.m_axi_num_write_outstanding`).

Controlling AXI4 Burst Behavior

An optimal AXI4 interface is one in which the design never stalls while waiting to access the bus, and after bus access is granted, the bus never stalls while waiting for the design to read/write. To create the optimal AXI4 interface, the following options are provided in the INTERFACE pragma or directive to specify the behavior of burst access and optimize the efficiency of the AXI4 interface. Refer to AXI Burst Transfers for more information on burst transfers.
In some cases, where autmatic burst access has failed, an efficient solution is to re-write the code or use manual burst as described in Using Manual Burst. If that does not work, then another solution might be to use cache memory in the AXI4 interface using the CACHE pragma or directive.

!! **Important:** A `volatile` qualifier on the variable prevents burst access to reado or write it.

Some of these options use internal storage to buffer data and can have an impact on area and resources:

- `latency`: Specifies the expected latency of the AXI4 interface, allowing the design to initiate a bus request a number of cycles (latency) before the read or write is expected. If this figure is too low, the design will be ready too soon and can stall waiting for the bus. If this figure is too high, bus access might be granted but the bus can stall waiting on the design to start the access.
- `max_read_burst_length`: Specifies the maximum number of data values read during a burst transfer.
- `num_read_outstanding`: Specifies how many read requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, a FIFO of size: `num_read_outstanding*max_read_burst_length*word_size`.
- `max_write_burst_length`: Specifies the maximum number of data values written during a burst transfer.
- `num_write_outstanding`: Specifies how many write requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, a FIFO of size: `num_write_outstanding*max_write_burst_length*word_size`

The following example can be used to help explain these options:

```
 #pragma HLS interface mode=m_axi port=input offset=slave bundle=gmem0
depth=1024*1024*16/(512/8)
 latency=100
 num_read_outstanding=32
 num_write_outstanding=32
 max_read_burst_length=16
 max_write_burst_length=16
```

The interface is specified as having a latency of 100. Vitis HLS seeks to schedule the request for burst access 100 clock cycles before the design is ready to access the AXI4 bus. To further improve bus efficiency, the options `num_write_outstanding` and `num_read_outstanding` ensure the design contains enough buffering to store up to 32 read and write bursts. This allows the design to continue processing until the bus requests are serviced. Finally, the options `max_read_burst_length` and `max_write_burst_length` ensure the maximum burst size is 16 and that the AXI4 interface does not hold the bus for longer than this.
These options allow the behavior of the AXI4 interface to be optimized for the system in which it will operate. The efficiency of the operation does depend on these values being set accurately.

Automatic Port Width Resizing

In the Vitis tool flow Vitis HLS provides the ability to automatically re-size `m_axi` interface ports to 512-bits to improve burst access. However, automatic port width resizing only supports standard C data types and power of two size `struct`, where the pointer is aligned to the expected widened byte size. If the tool cannot automatically widen the port, you can manually change the port width by using Vector or Arbitrary Precision (AP) as the data type of the port.

!! Important: Structs on the interface prevent automatic widening of the port. You must break the struct into individual elements to enable this feature.

Vitis HLS controls automatic port width resizing using the following two commands:

- `syn.interface.m_axi_max_widen_bitwidth=<N>`: Directs the tool to automatically widen bursts on M-AXI interfaces up to the specified bitwidth. The value of <N> must be a power-of-two between 0 and 1024.
- `syn.interface.m_axi_alignment_byte_size=<N>`: Note that burst widening also requires strong alignment properties. Assume pointers that are mapped to `m_axi` interfaces are at least aligned to the provided width in bytes (power of two). This can help automatic burst widening.

In the Vitis Kernel flow automatic port width resizing is enabled by default with the following:

```
syn.interface.m_axi_max_widen_bitwidth=512
syn.interface.m_axi_alignment_byte_size=64
```

In the Vivado IP flow this feature is disabled by default:

```
syn.interface.m_axi_max_widen_bitwidth=0
syn.interface.m_axi_alignment_byte_size=1
```

Automatic port width resizing will only re-size the port if a burst access can be seen by the tool. Therefore all the preconditions needed for bursting, as described in AXI Burst Transfers, are also needed for port resizing. These conditions include:

- Must be a monotonically increasing order of access (both in terms of the memory location being accessed as well as in time). You cannot access a memory location that is in between two previously accessed memory locations- aka no overlap.
- The access pattern from the global memory should be in sequential order, and with the following additional requirements:
  - The sequential accesses need to be on a primitive type or non vector power of two size aggregate type
  - The start of the sequential accesses needs to be aligned to the widen word size
  - The length of the sequential accesses needs to be divisible by the widen factor

The following code example is used in the calculations that follow:

```
vadd_pipeline:
  for (int i = 0; i < iterations; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_len/c_n max = c_len/c_n

  // Pipelining loops that access only one variable is the ideal way to
  // increase the global memory bandwidth.
  read_a:
    for (int x = 0; x < N; ++x) {
#pragma HLS LOOP_TRIPCOUNT min = c_n max = c_n
#pragma HLS PIPELINE II = 1
      result[x] = a[i * N + x];
    }

  read_b:
    for (int x = 0; x < N; ++x) {
#pragma HLS LOOP_TRIPCOUNT min = c_n max = c_n
#pragma HLS PIPELINE II = 1
      result[x] += b[i * N + x];
    }

  write_c:
    for (int x = 0; x < N; ++x) {
#pragma HLS LOOP_TRIPCOUNT min = c_n max = c_n
#pragma HLS PIPELINE II = 1
      c[i * N + x] = result[x];
    }
  }
}
}
```

The width of the automatic optimization for the code above is performed in three steps:

1. The tool checks for the number of access patterns in the read_a loop. There is one access during one loop iteration, so the optimization determines the interface bit-width as 32= 32 *1 (bitwidth of the int variable * accesses).
2. The tool tries to reach the default max specified by the `config_interface -m_axi_max_widen_bitwidth 512`, using the following expression terms:

   ```
   length = (ceil((loop-bound of index inner loops) *
   ```

```
(loop-bound of index - outer loops)) * #(of access-patterns))
```

- In the above code, the tool supports imperfect loop nest. If a and b are bundled to the same port, the tool will not extend the bursts on a and b due to conflicting. If a and b are bundled to different ports, the tool will extend the bursts on a and b to the outer loop. Therefore the formula will be shortened to:

  ```
  length = (ceil((loop-bound of index inner loops)) * #(of access-patterns))
  ```

  or: length = ceil(128) *32 = 4096

3. Is the calculated length a power of 2? If Yes, then the length will be capped to the width specified by `syn.interface.m_axi_max_widen_bitwidth`.

There are some pros and cons to using the automatic port width resizing which you should consider when using this feature. This feature improves the access throughput, instead of the data type size. It also adds more resources as it needs to buffer the huge vector and might need to shift the data accordingly to the data path size.

Creating an AXI4 Interface with 32-bit Address

By default, Vitis HLS implements the AXI4 port with a 64-bit address bus. However, some devices such as the Zynq 7000 have a 32 bit address bus. In this case you can implement the AXI4 interface with a 32-bit address bus by setting the `syn.interface.m_axi_addr64=0` configuration command to disable the 64-bit address bus.

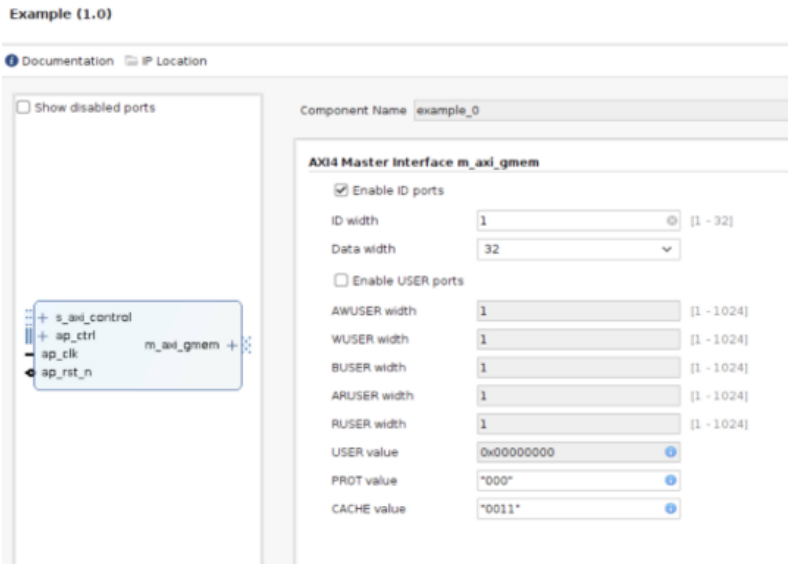
!! Important: When you disable the `syn.interface.m_axi_addr64` option, the HLS tool implements all AXI4 interfaces in the design with a 32-bit address bus.

Vitis HLS

Customizing AXI4 Master Interfaces in IP Integrator

When you incorporate an HLS RTL design that uses an AXI4 master interface into a design in the Vivado IP integrator, you can customize the block. From the block diagram in IP integrator, select the HLS block, right-click, and select Customize Block to customize any of the settings provided. A complete description of the AXI4 parameters are provided in the *Vivado Design Suite: AXI Reference Guide* (UG1037).
The following figure shows the Re-Customize IP dialog box for the design shown below. This design includes an AXI4-Lite port.

Figure: Customizing AXI4 Master Interfaces in IP Integrator



AXI4-Lite Interface

Overview

An HLS IP or kernel can be controlled by a host application, or embedded processor using the Slave AXI4-Lite interface (`s_axilite`) which acts as a system bus for communication between the processor and the kernel. Using the `s_axilite` interface the host or an embedded processor can start and stop the kernel, and read or write data to it. When Vitis HLS synthesizes the design the `s_axilite` interface is implemented as an adapter that captures the data that was communicated from the host in registers on the adapter. Refer to Vitis-HLS-Introductory-Examples/Interface/Register on GitHub for examples of some of these concepts.
The AXI4-Lite interface performs several functions within a Vivado IP or Vitis kernel:

- It maps a block-level control mechanism which can be used to start and stop the kernel.
- It provides a channel for passing scalar arguments, pointers to scalar values, function return values, and address offsets for `m_axi` interfaces from the host to the IP or kernel
- For the Vitis Kernel flow:
  - The tool will automatically infer the `s_axilite` interface pragma to provide offsets to pointer arguments assigned to `m_axi` interfaces, scalar values, and function return type.
  - Vitis HLS lets you read to or write from a pointer to a scalar value when assigned to an `s_axilite` interface. Pointers are assigned by default to `m_axi` interfaces, so this requires you to manually assign the pointer to the `s_axilite` using the INTERFACE pragma or directive:

    ```
    int top(int *a, int *b) {
    #pragma HLS interface s_axilite port=a
    ```

  - *Bundle*: Do not specify the `bundle` option for the `s_axilite` adapter in the Vitis Kernel flow. The tool will create a single `s_axilite` interface that will serve for the whole design.

    !! Important: HLS will return an error if multiple bundles are specified for the Vitis Kernel flow.
  - *Offset*: The tool will automatically choose the offsets for the interface. Do not specify any offsets in this flow.
- For the Vivado IP flow:
  - This flow will not use the `s_axilite` interface by default.
  - To use the `s_axilite` as a communication channel for scalar arguments, pointers to scalar values, offset to `m_axi` pointer address, and function return type, you must manually specify the INTERFACE pragma or directive.
  - *Bundle*: This flow supports multiple `s_axilite` interfaces, specified by bundle. Refer to S_AXILITE Bundle Rules for more information.
  - *Offset*: By default the tool will place the arguments in a sequential order starting from 0x10 in the control register map. Refer to S_AXILITE Offset Option for additional details.

S_AXILITE Example

The following example shows how Vitis HLS implements multiple arguments, including the function return, as an `s_axilite` interface. Because each pragma uses the same name for the `bundle` option, each of the ports is grouped into a single interface.

```
void example(char *a, char *b, char *c)
{
#pragma HLS INTERFACE mode=s_axilite port=return bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=a bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=b bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=c bundle=BUS_A
#pragma HLS INTERFACE mode=ap_vld port=b
```

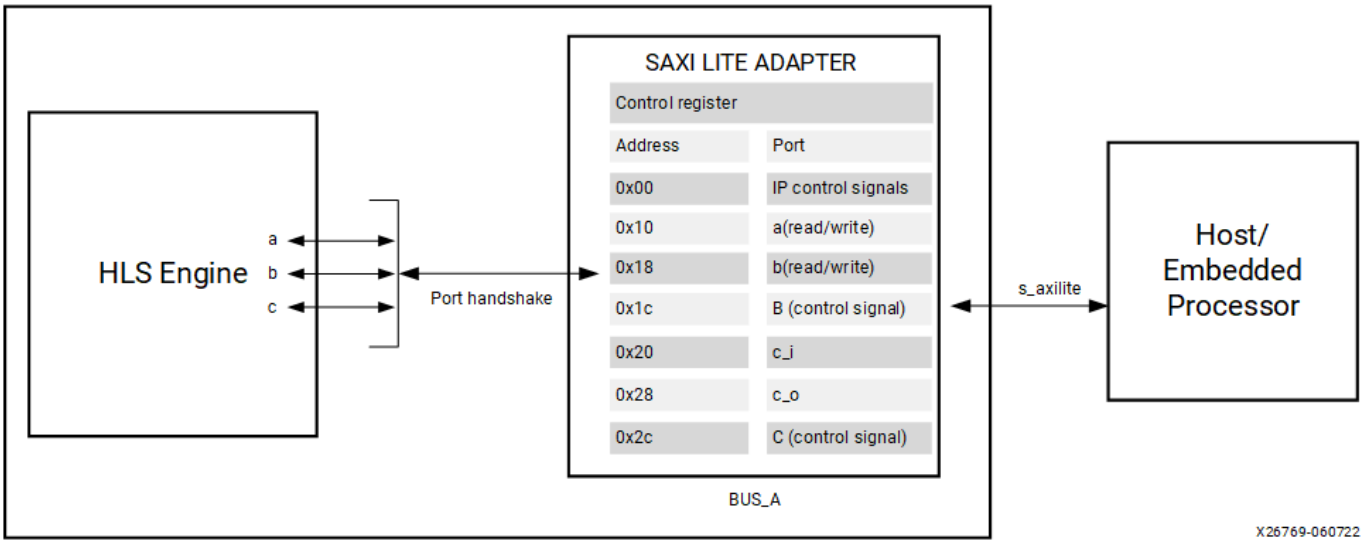
```
  *c += *a + *b;
}
```

★ **Tip:** If you do not specify the `bundle` option, Vitis HLS groups all arguments into a single `s_axilite` bundle and automatically names the port.

The synthesized example will be part of a system that has three important elements as shown in the figure below:

1. Host application running on an x86 or embedded processor interacting with the IP or kernel
2. SAXI Lite Adapter: The INTERFACE pragma implements an `s_axilite` adapter. The adapter has two primary functions: implementing the interface protocol to communicate with the host, and providing a Control Register Map to the IP or kernel.
3. The HLS engine or function that implements the design logic

Figure: S_AXILITE Adapter



By default, Vitis HLS automatically assigns the address for each port that is grouped into an `s_axilite` interface. The size, or range of addresses assigned to a port is dependent on the argument data type and the port protocol used, as described below. You can also explicitly define the address using the `offset` option as discussed in S_AXILITE Offset Option.

- Port a: By default, is implemented as `ap_none`. 1-word for the data signal is assigned and only 3 bits are used for addressing as the argument data type is `char`. Remaining bits are unused.
- Port b: is implemented as `ap_vld` defined by the INTERFACE pragma in the example. The corresponding control register is of size 2 bytes (16-bits) and is divided into two sections as follows:
  - (0x1c) Control signal : 1-word for the control signal is assigned.
  - (0x18) Data signal: 1-word for the data signal address is assigned and only 3 bits are used as the argument data type is `char`. Remaining bits are unused.
- Port c: By default, is implemented as `ap_ovld` as an output. The corresponding control register is of size 4 bytes (32 bits) and is divided into three sections:
  - (0x20) Data signal of `c_i`: 1-word for the input data signal is assigned, and only 3 bits are used for addressing as the argument data type is `char`, the rest are not used.
  - (0x24) Reserved Space
  - (0x28) Data signal of `c_o`: 1-word for the output data signal is assigned.
  - (0x2c) Control signal of `c_o` : 1-word for control signal `ap_ovld` is assigned and only 3 bits are used for addressing as the argument data type is `char`. Remaining bits are unused.

In operation the host application will initially start the kernel by writing into the Control address space (0x00). The host/CPU completes the initial setup by writing into the other address spaces which are associated with the various function arguments as defined in the example.

The control signal for port b is asserted and only then can the kernel read ports a and b (port a is ap_none and does not have a control signal). Until that time the design is stalled and waiting for the *valid* register to be set for port b. Each time port b is read by the HLS engine the input *valid* register is cleared and the register resets to logic 0.
After the HLS engine finishes its computation, the output value on port C is stored in the control register and the corresponding *valid* bit is set for the host to read. After the host reads the data, the HLS engine will write the ap_done bit in the Control register (0x00) to mark the end of the IP computation.
Vitis HLS reports the assigned addresses in the S_AXILITE Control Register Map, and also provides them in C Driver Files to aid in your software development. Using the s_axilite interface, you can exploit the C driver files for use with code running on an embedded or x86 processor using provided C application program interface (API) functions, to let you control the hardware from your software.

S_AXILITE Control Register Map

The Vitis unified IDE and v++ command automatically generates a Control Register Map for controlling the Vivado IP or Vitis kernel, and the ports grouped into s_axilite interface. The register map, which is added to the generated RTL files, can be divided into two sections:

1. Block-level control signals
2. Function arguments mapped into the s_axilite interface

In the Vitis kernel flow, the default block protocol is ap_ctrl_chain and is assigned to the s_axilite interface. The default settings should not be changed.
However, in the Vivado IP flow the default block control protocol is ap_ctrl_hs and is assigned to its own interface as seen in Interfaces for Vivado IP Flow. If you are using an s_axilite interface in your IP, you can also assign the block control protocol to that interface using the following INTERFACE pragma, as an example:

```
#pragma HLS INTERFACE mode=s_axilite port=return
```

To change the block control protocol you can also use the INTERFACE pragma or directive as follows:

```
#pragma HLS INTERFACE mode=ap_ctrl_chain port=return
```

In the Control Register Map of the s_axilite interface, Vitis HLS reserves addresses 0x00 through 0x18 for the block-level protocol, interrupt, mailbox and auto-restart controls. The latter are present only when counted auto-restart and the mailbox are enabled, as shown below:

Table: Addresses

Address	Description
0x00	Control signals
0x04	Global Interrupt Enable Register
0x08	IP Interrupt Enable Register (Read/Write)
0x0c	IP Interrupt Status Register (Read/TOW)
0x10	Auto-restart counter (Write; present only with counted autorestart)
0x14	Input mailbox write (Read/Write; present only when the input mailbox is enabled)
0x18	Output mailbox read (Read/Write; present only when the output mailbox is enabled)

The Control signals (0X00) contains `ap_start`, `ap_done`, `ap_ready`, and `ap_idle`; and in the case of `ap_ctrl_chain` the block protocol also contains `ap_continue`. These are the *block-level interface* signals which are accessed through the `s_axilite` adapter.
To start the block operation the`ap_start` bit in the Control register must be set to 1. The HLS engine will then proceed and read any inputs grouped into the AXI4-Lite slave interface from the register in the interface.
When the block completes the operation, the`ap_done`, `ap_idle` and `ap_ready` registers will be set by the hardware output ports and the results for any output ports grouped into the `s_axilite` interface read from the appropriate register.
For function arguments, the tool automatically assigns the address for each argument or port that is assigned to the `s_axilite` interface. The tool will assign each port an offset starting from `0x10`, the lower addresses being reserved for control signals. The size, or range of addresses assigned to a port is dependent on the argument data type and the port protocol used.
Because the variables grouped into an AXI4-Lite interface are function arguments which do not have a default value in the C code, none of the argument registers in the `s_axilite` interface can be assigned a default value. The registers can be implemented with a reset using the `config_rtl` command, but they cannot be assigned any other default value.
The Control Register Map generated by Vitis HLS for the `ap_ctrl_chain` block protocol is provided below:

```
...
//------------------------Address Info-------------------
// 0x00 : Control signals
//        bit 0  - ap_start (Read/Write/COH)
//        bit 1  - ap_done (Read)
//        bit 2  - ap_idle (Read)
//        bit 3  - ap_ready (Read/COR)
//        bit 4  - ap_continue (Read/Write/SC)
//        bit 7  - auto_restart (Read/Write)
//        bit 9  - interrupt (Read)
//        others - reserved
// 0x04 : Global Interrupt Enable Register
//        bit 0  - Global Interrupt Enable (Read/Write)
//        others - reserved
// 0x08 : IP Interrupt Enable Register (Read/Write)
//        bit 0 - enable ap_done interrupt (Read/Write)
//        bit 1 - enable ap_ready interrupt (Read/Write)
//        others - reserved
// 0x0c : IP Interrupt Status Register (Read/TOW)
//        bit 0 - ap_done (Read/TOW)
//        bit 1 - ap_ready (Read/TOW)
//        others - reserved
// 0x10 : Data signal of a
//        bit 7~0 - a[7:0] (Read/Write)
//        others  - reserved
// 0x14 : reserved
// 0x18 : Data signal of b
//        bit 7~0 - b[7:0] (Read/Write)
//        others  - reserved
// 0x1c : reserved
// 0x20 : Data signal of c_i
//        bit 7~0 - c_i[7:0] (Read/Write)
//        others  - reserved
// 0x24 : reserved
// 0x28 : Data signal of c_o
//        bit 7~0 - c_o[7:0] (Read)
//        others  - reserved
// 0x2c : reserved
```

```
// (SC = Self Clear, COR = Clear on Read, TOW = Toggle on Write, COH = Clear on Handshake)
...
```

S_AXILITE and Port-Level Protocols

Port-level I/O protocols sequence data into and out of the HLS engine from the `s_axilite` adapter as seen in S_AXILITE Example. In the Vivado IP flow, you can assign port-level I/O protocols to the individual ports and signals bundled into an `s_axilite` interface. In the Vitis kernel flow, changing the default port-level I/O protocols is not recommended unless necessary. The tool assigns a default port protocol to a port depending on the type and direction of the argument associated with it. The port can contain one or more of the following:

- Data signal for the argument
- Valid signal (`ap_vld`/`ap_ovld`) to indicate when the data can be read
- Acknowledge signal (`ap_ack`) to indicate when the data has been read

The default port protocol assignments for various argument types are as follows:

Table: Supported Argument Types

Argument Type	Default	Supported
scalar	`ap_none`	`ap_ack` and `ap_vld` can also be used
Pointers/References		
Inputs	`ap_none`	`ap_ack` and `ap_vld`
Outputs	`ap_vld`	`ap_none`, `ap_ack`, and `ap_ovld` can also be used
Inouts	`ap_ovld`	`ap_none`, `ap_ack`, and `ap_vld` are also supported

!! Important: Arrays default to `ap_memory`.

The S_AXILITE Example groups port `b` into the `s_axilite` interface and specifies port `b` as using the `ap_vld` protocol with INTERFACE pragmas. As a result, the `s_axilite` adapter contains a register for the port `b` data, and a register for the port `b` input valid signal.
If the input valid register is not set to logic 1, the data in the `b` data register is not considered valid, and the design stalls and waits for the valid register to be set. Each time port `b` is read, Vitis HLS automatically clears the input valid register and resets the register to logic 0.

Recommended: To simplify the operation of your design, AMD recommends that you use the default port protocols associated with the `s_axilite` interface.

S_AXILITE Bundle Rules

In the S_AXILITE Example all the function arguments are grouped into a single `s_axilite` interface adapter specified by the `bundle=BUS_A` option in the INTERFACE pragma. The `bundle` option simply lets you group ports together into one interface.
In the Vitis kernel flow there should only be a single interface bundle, commonly named `s_axi_control` by the tool. So you should not specify the `bundle` option in that flow, or you will probably encounter an error during synthesis. However, in the Vivado IP flow you can specify multiple bundles using the `s_axilite` interface, and this will create a separate interface adapter for each bundle you have defined. The following example shows this:

```
void example(char *a, char *b, char *c)
{
```

```
#pragma HLS INTERFACE mode=s_axilite port=a bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=b bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=c bundle=OUT
#pragma HLS INTERFACE mode=s_axilite port=return bundle=BUS_A
#pragma HLS INTERFACE mode=ap_vld port=b
  *c += *a + *b;
}
```

After synthesis completes, the Synthesis Summary report provides feedback regarding the number of `s_axilite` adapters generated. The SW-to-HW Mapping section of the report contains the HW info showing the control register offset and the address range for each port.
However, there are some rules related to using bundles with the `s_axilite` interface.

1. Default Bundle Names: This rule explicitly groups all interface ports with no bundle name into the same AXI4-Lite interface port, uses the tool default bundle name, and names the RTL port `s_axi_<default>`, typically `s_axi_control`.
   In this example all ports are mapped to the default bundle:

   ```
   void top(char *a, char *b, char *c)
   {
   #pragma HLS INTERFACE mode=s_axilite port=a
   #pragma HLS INTERFACE mode=s_axilite port=b
   #pragma HLS INTERFACE mode=s_axilite port=c
        *c += *a + *b;
   }
   ```

2. User-Specified Bundle Names: This rule explicitly groups all interface ports with the same `bundle` name into the same AXI4-Lite interface port, and names the RTL port the value specified by `s_axi_<string>`.
   The following example results in interfaces named `s_axi_BUS_A`, `s_axi_BUS_B`, and `s_axi_OUT`:

   ```
   void example(char *a, char *b, char *c)
   {
   #pragma HLS INTERFACE mode=s_axilite port=a bundle=BUS_A
   #pragma HLS INTERFACE mode=s_axilite port=b bundle=BUS_B
   #pragma HLS INTERFACE mode=s_axilite port=c bundle=OUT
   #pragma HLS INTERFACE mode=s_axilite port=return bundle=OUT
   #pragma HLS INTERFACE mode=ap_vld port=b
        *c += *a + *b;
   }
   ```

3. Partially Specified Bundle Names: If you specify `bundle` names for some arguments, but leave other arguments unassigned, then the tool will bundle the arguments as follows:
   - Group all ports into the specified bundles as indicated by the INTERFACE pragmas.
   - Group any ports without bundle assignments into a default named bundle. The default name can either be the standard tool default, or an alternative default name if the tool default has already been specified by the user.

   In the following example the user has specified `bundle=control`, which is the tool default name. In this case, port c will be assigned to `s_axi_control` as specified by the user, and the remaining ports will be bundled under `s_axi_control_r`, which is an alternative default name used by the tool.

   ```
   void top(char *a, char *b, char *c) {
   #pragma HLS INTERFACE mode=s_axilite port=a
   #pragma HLS INTERFACE mode=s_axilite port=b
   #pragma HLS INTERFACE mode=s_axilite port=c bundle=control
   }
   ```

S_AXILITE Offset Option

Note: The Vitis kernel flow determines the required offsets. Do not specify the `offset` option in that flow.

In the Vivado IP flow, Vitis HLS defines the size, or range of addresses assigned to a port in the S_AXILITE Control Register Map depending on the argument data type and the port protocol used. However, the INTERFACE pragma also contains an `offset` option that lets you specify the address offset in the AXI4-Lite interface.

When specifying the offset for your argument, you must consider the size of your data and reserve some extra for the port control protocol. The range of addresses you reserve should be based on a 32-bit word. You should reserve enough 32-bit words to fit your argument data type, and add reserve one additional word for the control protocol, even for `ap_none`.

Tip: In the case of the `ap_memory` protocol for arrays, you do not need to reserve the extra word for the control protocol. In this case, simply reserve enough 32-bit words to fit your argument data type.

For example, to reserve enough space for a double you need to reserve two 32-bit words for the 64-bit data type, and then reserve an additional 32-bit word for the control protocol. So you need to reserve a total of three 32-bit words, or 96 bits. If your argument offset starts at 0x020, then the next available offset would begin at 0x02c, in order to reserve the required address range for your argument.

If you make a mistake in setting the offset of your arguments, by not reserving enough address range to fit your data type and the control protocol, Vitis HLS will recognize the error, will warn you of the issue, and will recover by moving your misplaced argument register to the end of the Control Register Map. This will allow your build to proceed, but may not work with your host application or driver if they were written to your specified offset.

C Driver Files

When an AXI4-Lite slave interface is implemented, a set of C driver files are automatically created. These C driver files provide a set of APIs that can be integrated into any software running on a CPU and used to communicate with the device via the AXI4-Lite slave interface.

The C driver files are created when the design is packaged as IP in the IP catalog.

Driver files are created for standalone and Linux modes. In standalone mode the drivers are used in the same way as any other AMD standalone drivers. In Linux mode, copy all the C files (.c) and header files (.h) files into the software project.

The driver files and API functions derive their name from the top-level function for synthesis. In the above example, the top-level function is called “example”. If the top-level function was named “DUT” the name “example” would be replaced by “DUT” in the following description. The driver files are created in the packaged IP (located in the `impl` directory inside the solution).

Table: C Driver Files for a Design Named Example

File Path	Usage Mode	Description
data/example.mdd	Standalone	Driver definition file.
data/example.tcl	Standalone	Used by SDK to integrate the software into an SDK project.
src/xexample_hw.h	Both	Defines address offsets for all internal registers.
src/xexample.h	Both	API definitions
src/xexample.c	Both	Standard API implementations
src/xexample_sinit.c	Standalone	Initialization API implementations
src/xexample_linux.c	Linux	Initialization API implementations
src/Makefile	Standalone	Makefile

In file xexample.h, two structs are defined.

XExample_Config
: This is used to hold the configuration information (base address of each AXI4-Lite slave interface) of the IP instance.

XExample
: This is used to hold the IP instance pointer. Most APIs take this instance pointer as the first argument.

The standard API implementations are provided in files xexample.c, xexample_sinit.c, xexample_linux.c, and provide functions to perform the following operations.

- Initialize the device
- Control the device and query its status
- Read/write to the registers
- Set up, monitor, and control the interrupts

Refer to Vitis HLS C Driver Reference for a description of the API functions provided in the C driver files.

!! Important: The C driver APIs always use an unsigned 32-bit type (U32). You might be required to cast the data in the C code into the expected type.

C Driver Files and Float Types

C driver files always use a data 32-bit unsigned integer (U32) for data transfers. In the following example, the function uses float type arguments a and `r1`. It sets the value of a and returns the value of `r1`:

```
float calculate(float a, float *r1)
{
#pragma HLS INTERFACE mode=ap_vld register port=r1
#pragma HLS INTERFACE mode=s_axilite port=a
#pragma HLS INTERFACE mode=s_axilite port=r1
#pragma HLS INTERFACE mode=s_axilite port=return

 *r1 = 0.5f*a;
 return (a>0);
}
```

After synthesis, Vitis HLS groups all ports into the default AXI4-Lite interface and creates C driver files. However, as shown in the following example, the driver files use type U32:

```
// API to set the value of A
void XCalculate_SetA(XCalculate *InstancePtr, u32 Data) {
    Xil_AssertVoid(InstancePtr != NULL);
    Xil_AssertVoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);
    XCalculate_WriteReg(InstancePtr->Hls_periph_bus_BaseAddress,
XCALCULATE_HLS_PERIPH_BUS_ADDR_A_DATA, Data);
}

// API to get the value of R1
u32 XCalculate_GetR1(XCalculate *InstancePtr) {
    u32 Data;

    Xil_AssertNonvoid(InstancePtr != NULL);
    Xil_AssertNonvoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);

    Data = XCalculate_ReadReg(InstancePtr->Hls_periph_bus_BaseAddress,
XCALCULATE_HLS_PERIPH_BUS_ADDR_R1_DATA);
    return Data;
}
```

If these functions work directly with float types, the write and read values are not consistent with expected float type. When using these functions in software, you can use the following casts in the code:

```
float a=3.0f,r1;
u32 ua,ur1;

// cast float “a” to type U32
XCalculate_SetA(&calculate,*((u32*)&a));
ur1=XCalculate_GetR1(&calculate);

// cast return type U32 to float type for “r1”
r1=*((float*)&ur1);
```

Controlling Hardware

★ **Tip:** The example provided below demonstrates the `ap_ctrl_hs` block control protocol, which is the default for the Vivado IP flow. Refer to Block-Level Control Protocols for more information and a description of the `ap_ctrl_chain` protocol which is the default for the Vitis kernel flow.

In this example, the hardware header file `xexample_hw.h` provides a complete list of the memory mapped locations for the ports grouped into the AXI4-Lite slave interface, as described in S_AXILITE Control Register Map.

```
// 0x00 : Control signals
//        bit 0  - ap_start (Read/Write/SC)
//        bit 1  - ap_done (Read/COR)
//        bit 2  - ap_idle (Read)
//        bit 3  - ap_ready (Read)
//        bit 7  - auto_restart (Read/Write)
//        others - reserved
// 0x04 : Global Interrupt Enable Register
//        bit 0  - Global Interrupt Enable (Read/Write)
//        others - reserved
// 0x08 : IP Interrupt Enable Register (Read/Write)
//        bit 0  - Channel 0 (ap_done)
//        bit 1  - Channel 1 (ap_ready)
// 0x0c : IP Interrupt Status Register (Read/TOW)
//        bit 0  - Channel 0 (ap_done)
//        others - reserved
// 0x10 : Data signal of a
//        bit 7~0 - a[7:0] (Read/Write)
//        others  - reserved
// 0x14 : reserved
// 0x18 : Data signal of b
//        bit 7~0 - b[7:0] (Read/Write)
//        others  - reserved
// 0x1c : reserved
// 0x20 : Data signal of c_i
//        bit 7~0 - c_i[7:0] (Read/Write)
//        others  - reserved
// 0x24 : reserved
// 0x28 : Data signal of c_o
//        bit 7~0 - c_o[7:0] (Read)
//        others  - reserved
// 0x2c : Control signal of c_o
//        bit 0  - c_o_ap_vld (Read/COR)
//        others - reserved
```

```
// (SC = Self Clear, COR = Clear on Read, TOW = Toggle on Write, COH = Clear on
Handshake)
```

To correctly program the registers in the `s_axilite` interface, you must understand how the hardware ports operate with the default port protocols, or the custom protocols as described in S_AXILITE and Port-Level Protocols.
For example, to start the block operation the `ap_start` register must be set to 1. The device will then proceed and read any inputs grouped into the AXI4-Lite slave interface from the register in the interface. When the block completes operation, the `ap_done`, `ap_idle` and `ap_ready` registers will be set by the hardware output ports and the results for any output ports grouped into the AXI4-Lite slave interface read from the appropriate register.
The implementation of function argument `c` in the example highlights the importance of some understanding how the hardware ports operate. Function argument `c` is both read and written to, and is therefore implemented as separate input and output ports `c_i` and `c_o`, as explained in S_AXILITE Example.
The first recommended flow for programing the `s_axilite` interface is for a one-time execution of the function:

- Use the interrupt function standard API implementations provided in the C Driver Files to determine how you want the interrupt to operate.
- Load the register values for the block input ports. In the above example this is performed using API functions `XExample_Set_a`, `XExample_Set_b`, and `XExample_Set_c_i`.
- Set the `ap_start` bit to 1 using `XExample_Start` to start executing the function. This register is self-clearing as noted in the header file above. After one transaction, the block will suspend operation.
- Allow the function to execute. Address any interrupts which are generated.
- Read the output registers. In the above example this is performed using API functions `XExample_Get_c_o_vld`, to confirm the data is valid, and `XExample_Get_c_o`.

  Note: The registers in the `s_axilite` interface obey the same I/O protocol as the ports. In this case, the output valid is set to logic 1 to indicate if the data is valid.
- Repeat for the next transaction.

The second recommended flow is for continuous execution of the block. In this mode, which is described in much more detail in the next section, the input ports included in the AXI4-Lite interface should only be ports which perform configuration. The block will typically run much faster than a CPU. If the block must wait for inputs, the block will spend most of its time waiting:

- Use the interrupt function to determine how you wish the interrupt to operate.
- Load the register values for the block input ports. In the above example this is performed using API functions `XExample_Set_a`, `XExample_Set_a` and `XExample_Set_c_i`.
- Set the auto-start function using API `XExample_EnableAutoRestart`.
- Allow the function to execute. The individual port I/O protocols will synchronize the data being processed through the block.
- Address any interrupts which are generated. The output registers could be accessed during this operation but the data may change often.
- Use the API function `XExample_DisableAutoRestart` to prevent any more executions.
- Read the output registers. In the above example this is performed using API functions `XExample_Get_c_o` and `XExample_Set_c_o_vld`.

Controlling Software

The API functions can be used in the software running on the CPU to control the hardware block. An overview of the process is:

- Create an instance of the hardware
- Look Up the device configuration
- Initialize the device
- Set the input parameters of the HLS block
- Start the device and read the results

An example application is shown below.

```
#include "xexample.h"    // Device driver for HLS HW block
#include "xparameters.h"

// HLS HW instance
XExample HlsExample;
XExample_Config *ExamplePtr

int main() {
 int res_hw;

// Look Up the device configuration
 ExamplePtr = XExample_LookupConfig(XPAR_XEXAMPLE_0_DEVICE_ID);
 if (!ExamplePtr) {
 print("ERROR: Lookup of accelerator configuration failed.\n\r");
 return XST_FAILURE;
 }

// Initialize the Device
 status = XExample_CfgInitialize(&HlsExample, ExamplePtr);
 if (status != XST_SUCCESS) {
 print("ERROR: Could not initialize accelerator.\n\r");
 exit(-1);
 }

//Set the input parameters of the HLS block
 XExample_Set_a(&HlsExample, 42);
 XExample_Set_b(&HlsExample, 12);
 XExample_Set_c_i(&HlsExample, 1);

// Start the device and read the results
 XExample_Start(&HlsExample);
 do {
 res_hw = XExample_Get_c_o(&HlsExample);
 } while (XExample_Get_c_o(&HlsExample) == 0); // wait for valid data output
 print("Detected HLS peripheral complete. Result received.\n\r");
}
```

Control Clock and Reset in AXI4-Lite Interfaces

Note: If you instantiate the slave AXI4-Lite register file in a bus fabric that uses a different clock frequency, Vivado IP integrator will automatically generate a clock domain crossing (CDC) slice that performs the same function as the control clock described below, making use of the option unnecessary.

By default, Vitis HLS uses the same clock for the AXI4-Lite interface and the synthesized design. Vitis HLS connects all registers in the AXI4-Lite interface to the clock used for the synthesized logic (`ap_clk`).
Optionally, you can use the INTERFACE directive `clock` option to specify a separate clock for each AXI4-Lite port. When connecting the clock to the AXI4-Lite interface, you must use the following protocols:

- AXI4-Lite interface clock must be synchronous to the clock used for the synthesized logic (`ap_clk`). That is, both clocks must be derived from the same master generator clock.
- AXI4-Lite interface clock frequency must be equal to or less than the frequency of the clock used for the synthesized logic (`ap_clk`).

If you use the `clock` option with the INTERFACE directive, you only need to specify the `clock` option on one function argument in each bundle. Vitis HLS implements all other function arguments in the bundle with the same clock and reset.

Vitis HLS names the generated reset signal with the prefix `ap_rst_` followed by the clock name. The generated reset signal is active-Low independent of the `config_rtl` command.
The following example shows how Vitis HLS groups function arguments `a` and `b` into an AXI4-Lite port with a clock named `AXI_clk1` and an associated reset port.

```
// Default AXI-Lite interface implemented with independent clock called AXI_clk1
#pragma HLS interface mode=s_axilite port=a clock=AXI_clk1
#pragma HLS interface mode=s_axilite port=b
```

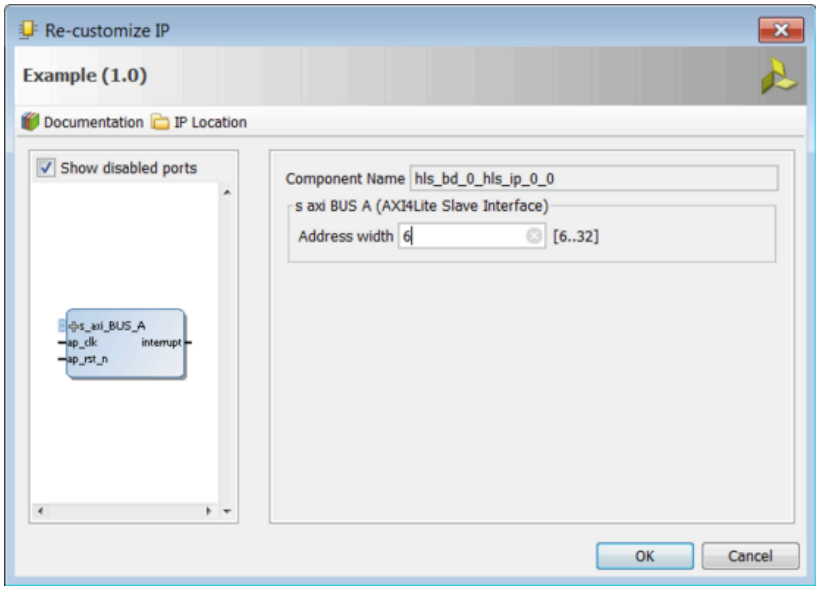
In the following example, Vitis HLS groups function arguments `c` and `d` into AXI4-Lite port `CTRL1` with a separate clock called `AXI_clk2` and an associated reset port.

```
// CTRL1 AXI-Lite bundle implemented with a separate clock (called AXI_clk2)
#pragma HLS interface mode=s_axilite port=c bundle=CTRL1 clock=AXI_clk2
#pragma HLS interface mode=s_axilite port=d bundle=CTRL1
```

Customizing AXI4-Lite Slave Interfaces in IP Integrator

When an HLS RTL design using an AXI4-Lite slave interface is incorporated into a design in Vivado IP integrator, you can customize the block. From the block diagram in IP integrator, select the HLS block, right-click with the mouse button and select Customize Block.
The address width is by default configured to the minimum required size. Modify this to connect to blocks with address sizes less than 32-bit.

Figure: Customizing AXI4-Lite Slave Interfaces in IP Integrator



AXI4-Stream Interfaces

!! Important: `hls::axis` (and `ap_axiu`/`ap_axis`) cannot be used on internal functions or variables as the AXI4-Stream protocol is only supported on the interfaces of top-level functions. For internal functions or variables you must use `hls::stream` objects as described in HLS Stream Library.

An AXI4-Stream interface can be applied to any input argument and any array or pointer output argument. Because an AXI4-Stream interface transfers data in a sequential streaming manner, it cannot be used with arguments that are both read and written. In terms of data layout, the data type of the AXI4-Stream is aligned to the next byte. For example, if the size of the data type is 12 bits, it will be extended to 16 bits. Depending on whether a signed/unsigned interface is selected, the extended bits are either sign-extended or zero-extended.

If the stream data type is an user-defined struct, the default procedure is to keep the struct aggregated and align the struct to the size of the largest data element to the nearest byte. The only exception to this rule is if the struct contains a `hls::stream` object. In this special case, the struct will be disaggregated and an axi stream will be created for each member element of the struct.

★ **Tip:** The maximum supported port width is 4096 bits, even for aggregated structs or reshaped arrays.

The following code examples show how the packed alignment depends on your struct type. If the struct contains only char type, as shown in the following example, then it will be packed with alignment of one byte. Total size of the struct will be two bytes:

```
struct A {
  char foo;
  char bar;
};
```

However, if the struct has elements with different data types, as shown below, then it will be packed and aligned to the size of the largest data element, or four bytes in this example. Element `bar` will be padded with three bytes resulting in a total size of eight bytes for the struct:

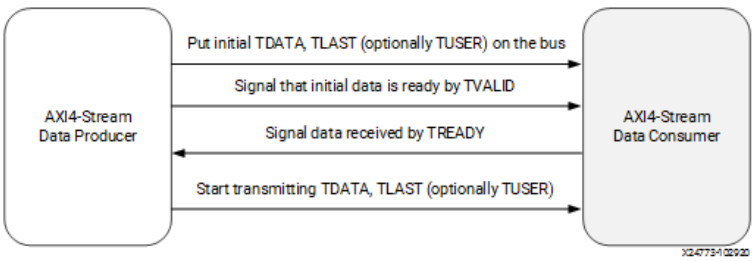
```
struct A {
  int foo;
  char bar;
};
```

!! **Important:** Structs contained in AXI4-Stream interfaces (axis) are aggregated by default, and the stream itself cannot be disaggregated. If separate streams for member elements of the struct are desired then this must be manually coded as separate elements, resulting in a separate axis interface for each element. Refer to Vitis-HLS-Introductory-Examples/Interface/Aggregation_Disaggregation/disaggregation_of_axis_port on GitHub for an example.

How AXI4-Stream Works

AXI4-Stream is a protocol designed for transporting arbitrary unidirectional data. In an AXI4-Stream, `TDATA` width of bits is transferred per clock cycle. The transfer is started after the producer sends the `TVALID` signal and the consumer responds by sending the `TREADY` signal (after it has consumed the initial `TDATA`). At this point, the producer will start sending `TDATA` and `TLAST` (`TUSER` if needed to carry additional user-defined side-channel data). `TLAST` signals the last byte of the stream. So the consumer keeps consuming the incoming `TDATA` until `TLAST` is asserted.

Figure: AXI4-Stream Handshake



AXI4-Stream has additional optional features like sending positional data with `TKEEP` and `TSTRB` ports which makes it possible to multiplex both the data position and data itself on the `TDATA` signal. Using the `TID` and `TDIST` signals, you can route streams as these fields roughly corresponds to stream identifier and stream destination identifier. Refer to *Vivado Design Suite: AXI Reference Guide* (UG1037) or the *AMBA AXI4-Stream Protocol Specification* (ARM IHI 0051A) for more information.

If your design requires a streaming interface begin by defining and using a streaming data structure like `hls::stream` in Vitis HLS. This simple object encapsulates the requirements of streaming and its streaming interface is by default implemented in the RTL as a FIFO interface (ap_fifo) but can be optionally, implemented as a handshake interface (ap_hs) or an AXI4-Stream interface (axis). Refer to Vitis-HLS-Introductory-Examples/Interface/Streaming on GitHub for different examples of streaming interfaces.
If a AXI4-Stream interface (axis) is specified via the interface pragma mode option, the interface implementation will mimic the style of an AXIS interface by defining the TDATA, TVALID and TREADY signals.
If a more formal AXIS implementation is desired, then Vitis HLS requires the usage of a special data type (`hls::axis` defined in `ap_axi_sdata.h`) to encapsulate the requirements of the AXI4-Stream protocol and implement the special RTL signals needed for this interface.
The AXI4-Stream interface is implemented as a struct type in Vitis HLS and has the following signature (defined in ap_axi_sdata.h):

```
template <typename T, size_t WUser, size_t WId, size_t WDest> struct axis { .. };
```

Where:

`T`
: The data type to be streamed.

  ★ **Tip:** This can support any data type, including `ap_fixed`.

`WUser`
: Width of the TUSER signal

`WId`
: Width of the TID signal

`WDest`
: Width of the TDest signal

When the stream data type (`T`) are simple integer types, there are two predefined types of AXI4-Stream implementations available:

- A signed implementation of the AXI4-Stream class (or more simply `ap_axis<Wdata, WUser, WId, WDest>`)

  ```
  hls::axis<ap_int<WData>, WUser, WId, WDest>
  ```

- An unsigned implementation of the AXI4-Stream class (or more simply `ap_axiu<WData, WUser, WId, WDest>`)

  ```
  hls::axis<ap_uint<WData>, WUser, WId, WDest>
  ```

The value specified for the `WUser`, `WId`, and `WDest` template parameters controls the usage of side-channel signals in the AXI4-Stream interface.
When the `hls::axis` class is used, the generated RTL will typically contain the actual data signal `TDATA`, and the following additional signals: `TVALID`, `TREADY`, `TKEEP`, `TSTRB`, `TLAST`, `TUSER`, `TID`, and `TDEST`.
`TVALID`, `TREADY`, and `TLAST` are necessary control signals for the AXI4-Stream protocol. `TKEEP`, `TSTRB`, `TUSER`, `TID`, and `TDEST` signals are optional special signals that can be used to pass around additional bookkeeping data.

★ **Tip:** If `WUser`, `WId`, and `WDest` are set to 0, the generated RTL will not include the optional `TUSER`, `TID`, and `TDEST` signals in the interface.

Registered AXI4-Stream Interfaces

As a default, AXI4-Stream interfaces are always implemented as registered interfaces to ensure that no combinational feedback paths are created when multiple HLS IP blocks with AXI4-Stream interfaces are integrated into a larger design. For AXI4-Stream interfaces, four types of register modes are provided to control how the interface registers are implemented:

Forward
: Only the `TDATA` and `TVALID` signals are registered.

Reverse
: Only the `TREADY` signal is registered.

Both
: All signals (`TDATA`, `TREADY`, and `TVALID`) are registered. This is the default.

Off
: None of the port signals are registered.

The AXI4-Stream side-channel signals are considered to be data signals and are registered whenever `TDATA` is registered.

Recommended: When connecting HLS generated IP blocks with AXI4-Stream interfaces at least one interface should be implemented as a registered interface or the blocks should be connected via an AXI4-Stream Register Slice.

There are two basic methods to use an AXI4-Stream in your design:

- Use an AXI4-Stream without side-channels.
- Use an AXI4-Stream with side-channels.

This second use model provides additional functionality, allowing the optional side-channels which are part of the AXI4-Stream standard, to be used directly in your C/C++ code.

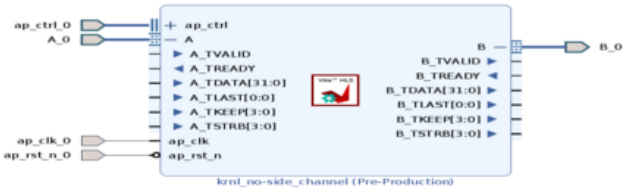
AXI4-Stream Interfaces without Side-Channels

An AXI4-Stream is used without side-channels when the function argument, `ap_axis` or `ap_axiu` data type, does not contain any AXI4 side-channel elements (that is, when the `WUser`, `WId`, and `WDest` parameters are set to 0). In the following example, both interfaces are implemented using an AXI4-Stream:

```
#include "ap_axi_sdata.h"
#include "hls_stream.h"

typedef ap_axiu<32, 0, 0, 0> trans_pkt;

void example(hls::stream< trans_pkt > &A, hls::stream< trans_pkt > &B)
{
#pragma HLS INTERFACE mode=axis port=A
#pragma HLS INTERFACE mode=axis port=B
    trans_pkt tmp;
    A.read(tmp);
    tmp.data += 5;
    B.write(tmp);
}
```

After synthesis, both arguments are implemented with a data port (`TDATA`) and the standard AXI4-Stream protocol ports, `TVALID`, `TREADY`, `TKEEP`, `TLAST`, and `TSTRB`, as shown in the following figure.

Figure: AXI4-Stream Interfaces without Side-Channels



★ **Tip:** If you specify an `hls::stream` object with a data type other than `ap_axis` or `ap_axiu`, the tool will infer an AXI4-Stream interface without the `TLAST` signal, or any of the side-channel signals. This implementation of the AXI4-Stream interface consumes fewer device resources, but offers no visibility into when the stream is ending.

Multiple variables can be combined into the same AXI4-Stream interface by using a struct, which is aggregated by Vitis HLS by default. Aggregating the elements of a struct into a single wide-vector, allows all elements of the struct to be implemented in the same AXI4-Stream interface.

AXI4-Stream Interfaces with Side-Channels

The following two example show how the side-channels can be used directly in the C/C++ code and implemented on the interface. The code uses #include "ap_axi_sdata.h" to provide an API to handle the side-channels of the AXI4-Stream interface.
The first example uses recommended APIs with getters/setters listed below while the second example uses the legacy coding style.

```
Table for getters/setters APIs:
Getters Setters
get_data()      set_data()
get_keep()      set_keep()
get_strb()      set_strb()
get_user()      set_user()
get_last()      set_last()
get_id()        set_id()
get_dest()      set_dest()


First example:
#include "ap_axi_sdata.h"
#include "ap_int.h"
#include "hls_stream.h"

#define DWIDTH 32
typedef ap_axiu<DWIDTH, 1, 1, 1> trans_pkt;

extern "C"
{
    void krnl_stream_vmult(hls::stream<trans_pkt> &A,
                           hls::stream<trans_pkt> &B)
    {
#pragma HLS INTERFACE mode = axis port = A
#pragma HLS INTERFACE mode = axis port = B
#pragma HLS INTERFACE mode = s_axilite port = return bundle = control
        bool eos = false;
    vmult: do
        {
#pragma HLS PIPELINE II = 1
            trans_pkt t2 = A.read();

            // Packet for Output
```

```
            trans_pkt t_out;

            // Reading data from input packet
            ap_uint<DWIDTH> in2 = t2.get_data();
            ap_uint<DWIDTH> tmpOut = in2 * 5;

            // Setting data and configuration to output packet
            t_out.set_data(tmpOut);
            t_out.set_last(t2.get_last());
            t_out.set_keep(-1); // Enabling all bytes
            // Writing packet to output stream
            B.write(t_out);
            if (t2.get_last())
            {
                eos = true;
            }
        } while (eos == false);
    }
}
```

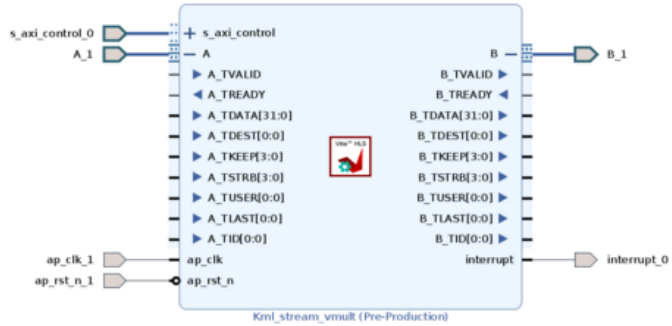
The second example with legacy coding style:

```
#include "ap_axi_sdata.h"
#include "ap_int.h"
#include "hls_stream.h"

#define DWIDTH 32

typedef ap_axiu<DWIDTH, 1, 1, 1> trans_pkt;

extern "C"{
    void krnl_stream_vmult(hls::stream<trans_pkt> &A,
                                                    hls::stream<trans_pkt> &B) {
#pragma HLS INTERFACE mode=axis port=A
#pragma HLS INTERFACE mode=axis port=B
#pragma HLS INTERFACE mode=s_axilite port=return bundle=control
        bool eos = false;

        vmult: do {
#pragma HLS PIPELINE II=1
            trans_pkt t2 = A.read();

            // Packet for Output
            trans_pkt t_out;

            // Reading data from input packet
            ap_uint<DWIDTH> in2 = t2.data;
            ap_uint<DWIDTH> tmpOut = in2 * 5;

            // Setting data and configuration to output packet
            t_out.data = tmpOut;
            t_out.last = t2.last;
            t_out.keep = -1; //Enabling all bytes
            // Writing packet to output stream
            B.write(t_out);
            if (t2.last) {
               eos = true;
```

```
            }
        } while (eos == false);
    }
}
```

After synthesis, both the A and B arguments are implemented with data ports, the standard AXI4-Stream protocol ports, `TVALID` and `TREADY` and all of the optional ports described in the struct.

Figure: AXI4-Stream Interfaces with Side-Channels



Coding Style for Array to Stream

While arrays can be converted to streams, it can often lead to coding and synthesis issues as arrays can be accessed in random order while a stream requires a sequential access pattern where every element is read in order. To avoid such issues, any time a streaming interface is required, it is highly recommended to use the `hls::stream` object as described in Using HLS Streams. Usage of this construct will enforce streaming semantics in the source code.
However, to convert an array to a stream you should perform all the operations on temp variables. Read the input stream, process the temp variable, and write the output stream, as shown in the example below. This approach lets you preserve the sequential reading and writing of the stream of data, rather than attempting multiple or random reads or writes.

```
struct A {
  short varA;
  int varB;
};

void dut(A in[N], A out[N], bool flag) {
#pragma HLS interface mode=axis port=in,out
   for (unsigned i=0; i<N; i++) {
      A tmp = in[i];
      if (flag)
         tmp.varB = tmp.varA + 5;
   out[i] = tmp;
   }
}
```

If this coding style is not adhered to, it will lead to functional failures of the stream processing.
The recommended method is to define the arguments as `hls::stream` objects as shown below:

```
void dut(hls::stream<A> &in, hls::stream<A> &out, bool flag) {
#pragma HLS interface mode=axis port=in,out

for (unsigned i=0; i<N; i++) {
   A tmp = in.read();
   if (flag)
      tmp.varB = tmp.varA + 5;
```

```
    out.write(tmp);
  }
}
```

Customizing AXI4-Stream Interfaces

An instance of an AXI4-Stream without side channels (for example `hls::stream<ap_axiu<32, 0, 0, 0>>`) can result in the following RTL signals on the interface:

- TVALID
- TREADY
- TDATA[ ]
- TLAST[ ]
- TKEEP[ ]
- TSTRB[ ]

Some of these signals (TLAST, TKEEP, and TSTRB) can be unnecessary or unwanted in your design. In addition, there are use cases where TDATA is not required as the modeler uses TUSER to move data. To support these additional use cases the `hls::axis<T>` definition supports the following options:

```
template <typename T,
          std::size_t WUser = 0,
          std::size_t WId = 0,
          std::size_t WDest = 0,
          uint8_t EnableSignals = (AXIS_ENABLE_KEEP |
                                   AXIS_ENABLE_LAST |
                                   AXIS_ENABLE_STRB),
          bool StrictEnablement = false>
struct axis {
  // members
  T data;
  ap_uint<bytesof<T>> strb;
  ap_uint<bytesof<T>> keep;
  ap_uint<WUser>      user;
  ap_uint<WId>        id;
  ap_uint<WDest>      dest;
  ap_uint<1>          last;

  // get_* methods
  …
  // set_* methods
  …
};
```

The EnableSignals and StrictEnablement fields let you specify which of the three signals (TKEEP, TLAST, and TSTRB) should appear in your design using the specification of special macros. The complete list of macros is shown below:

```
// Enablement for axis signals
#define AXIS_ENABLE_DATA 0b00000001
#define AXIS_ENABLE_DEST 0b00000010
#define AXIS_ENABLE_ID   0b00000100
#define AXIS_ENABLE_KEEP 0b00001000
#define AXIS_ENABLE_LAST 0b00010000
#define AXIS_ENABLE_STRB 0b00100000
#define AXIS_ENABLE_USER 0b01000000
```

```
// Disablement mask for DATA axis signals
#define AXIS_DISABLE_DATA (0b11111111 ^ AXIS_ENABLE_DATA) & \
                          (0b11111111 ^ AXIS_ENABLE_KEEP) & \
                          (0b11111111 ^ AXIS_ENABLE_STRB)

// Enablement/disablement of all axis signals
#define AXIS_ENABLE_ALL  0b01111111
#define AXIS_DISABLE_ALL 0b00000000
```

★ **Tip:** These macros also enable some error checking to identify cases where your specification is incomplete.

In addition to letting you enable or disable specific side-channel signals, the set of macros was expanded to allow the specification of `all` or none of the signals. This allowed for the definition of the special `hls::axis_data<>` and `hls::axis_user<>` helper classes that make it easier to pick and choose the exact signal specification. The definitions of these special helper classes are shown below:

```
// Struct: axis_data (alternative to axis)
//   DATA signal always enabled
//   All other signals are optional, disabled by default
template <typename TData,
          uint8_t EnableSignals = AXIS_ENABLE_DATA,
          std::size_t WUser = 0,
          std::size_t WId = 0,
          std::size_t WDest = 0,
          bool StrictEnablement = true>
using axis_data = axis<TData, WUser, WId, WDest,
                       (EnableSignals | AXIS_ENABLE_DATA),
                       StrictEnablement>;

// Struct: axis_user (alternative to axis)
//   USER signal always enabled
//   DATA signal always disabled
//   All other signals are optional, disabled by default
template <std::size_t WUser,
          uint8_t EnableSignals = AXIS_ENABLE_USER,
          std::size_t WId = 0,
          std::size_t WDest = 0,
          bool StrictEnablement = true>
using axis_user = axis<void, WUser, WId, WDest,
                       (EnableSignals & AXIS_DISABLE_DATA),
                       StrictEnablement>;
```

The following use cases illustrate the various options letting you customize the signals associated with the AXI4-Stream interface:

```
// Allow user to control existence of TKEEP, TSTRB and TLAST by template parameter
`EnableSideChannels`

// Enable only TKEEP
using data_t = hls::axis<int, 0, 0, 0, AXIS_ENABLE_KEEP>; Or using data_t = hls::axis_data<int,
AXIS_ENABLE_KEEP>

// Enable two channels TKEEP and TLAST
using data_t = hls::axis_data<int, AXIS_ENABLE_KEEP|AXIS_ENABLE_LAST>;

// Disable all side-channels
using data_t = hls::axis_data<int, AXIS_DISABLE_ALL>;
```

```
// Enable all side-channels
using data_t = hls::axis_data<int, AXIS_ENABLE_ALL, 4, 2, 3>;

// Error case: All signals enabled but zero size specified for WUser/WId/WDest
using data_t = hls::axis_data<int, AXIS_ENABLE_ALL>;

// Allow user to control existence of TDATA by specifying template parameter `T` with `void` type
// and only use TUSER channel
using data_t = hls::axis<void, 5, 0, 0, AXIS_DISABLE_ALL>; Or using data_t = hls::axis_user<5>

// Use default enabling of TDATA, TKEEP, TSTRB and TLAST
using data_t = hls::axis<int>;
```

Port-Level Protocols for Vivado IP Flow

!! Important: The port-level protocols described here can be used in the Vivado IP flow as explained in Interfaces for Vivado IP Flow. These protocols are not supported in the Vitis kernel flow.

By default input pointers and pass-by-value arguments are implemented as simple wire ports with no associated handshaking signal. For example, in the `vadd` function discussed in Interfaces for Vivado IP Flow, the input ports are implemented without an I/O protocol, only a data port. If the port has no I/O protocol, (by default or by design) the input data must be held stable until it is read.

By default output pointers are implemented with an associated output valid signal to indicate when the output data is valid. In the `vadd` function example, the output port is implemented with an associated output valid port (`out_r_o_ap_vld`) which indicates when the data on the port is valid and can be read. If there is no I/O protocol associated with the output port, it is difficult to know when to read the data.

★ Tip: It is always a good idea to use an I/O protocol on an output.

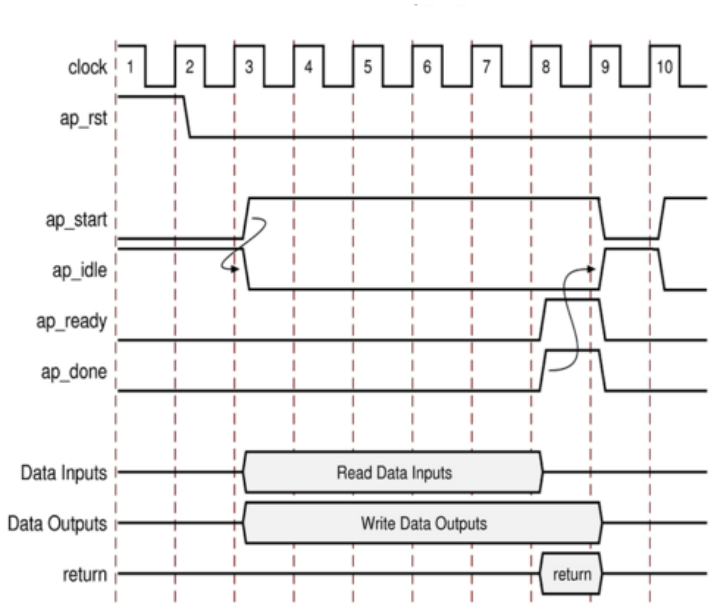
Function arguments which are both read from and written to are split into separate input and output ports. In the `vadd` function example, the `out_r` argument is implemented as both an input port `out_r_i`, and an output port `out_r_o` with associated I/O protocol port `out_r_o_ap_vld`.

If the function has a return value, an output port `ap_return` is implemented to provide the return value. When the RTL design completes one transaction, this is equivalent to one execution of the C/C++ function, the block-level protocols indicate the function is complete with the `ap_done` signal. This also indicates the data on port `ap_return` is valid and can be read.

Note: The return value of the top-level function cannot be a pointer.

For the example code shown the timing behavior is shown in the following figure (assuming that the target technology and clock frequency allow a single addition per clock cycle).

Figure: RTL Port Timing with Default Synthesis



- The design starts when `ap_start` is asserted High.
- The `ap_idle` signal is asserted Low to indicate the design is operating.
- The input data is read at any clock after the first cycle. Vitis HLS schedules when the reads occur. The `ap_ready` signal is asserted High when all inputs have been read.
- When output `sum` is calculated, the associated output handshake (`sum_o_ap_vld`) indicates that the data is valid.
- When the function completes, `ap_done` is asserted. This also indicates that the data on `ap_return` is valid.
- Port `ap_idle` is asserted High to indicate that the design is waiting start again.

Port-Level I/O: No Protocol

The `ap_none` specifies that no I/O protocol be added to the port. When this is specified the argument is implemented as a data port with no other associated signals. The `ap_none` mode is the default for scalar inputs.

ap_none

The `ap_none` port-level I/O protocol is the simplest interface type and has no other signals associated with it. Neither the input nor output data signals have associated control ports that indicate when data is read or written. The only ports in the RTL design are those specified in the source code.
An `ap_none` interface does not require additional hardware overhead. However, the `ap_none` interface does requires the following:

- Producer blocks to do one of the following:
  - Provide data to the input port at the correct time, typically before the design starts.
  - Hold data for the length of a transaction until the design raises the ap_ready signal.
- Consumer blocks to read output ports when the design is done, and before it is started again.

Note: The `ap_none` interface cannot be used with array arguments.

Port-Level I/O: Wire Handshakes

Interface mode `ap_hs` includes a two-way handshake signal with the data port. The handshake is an industry standard valid and acknowledge handshake. Mode `ap_vld` is the same but only has a valid port and `ap_ack` only has a acknowledge port.
Mode `ap_ovld` is for use with in-out arguments. When the in-out is split into separate input and output ports, mode `ap_none` is applied to the input port and `ap_vld` applied to the output port. This is the default for pointer arguments that are both read and written.

The ap_hs mode can be applied to arrays that are read or written in sequential order. If Vitis HLS can determine the read or write accesses are not sequential, it will halt synthesis with an error. If the access order cannot be determined, Vitis HLS will issue a warning.

ap_hs (ap_ack, ap_vld, and ap_ovld)

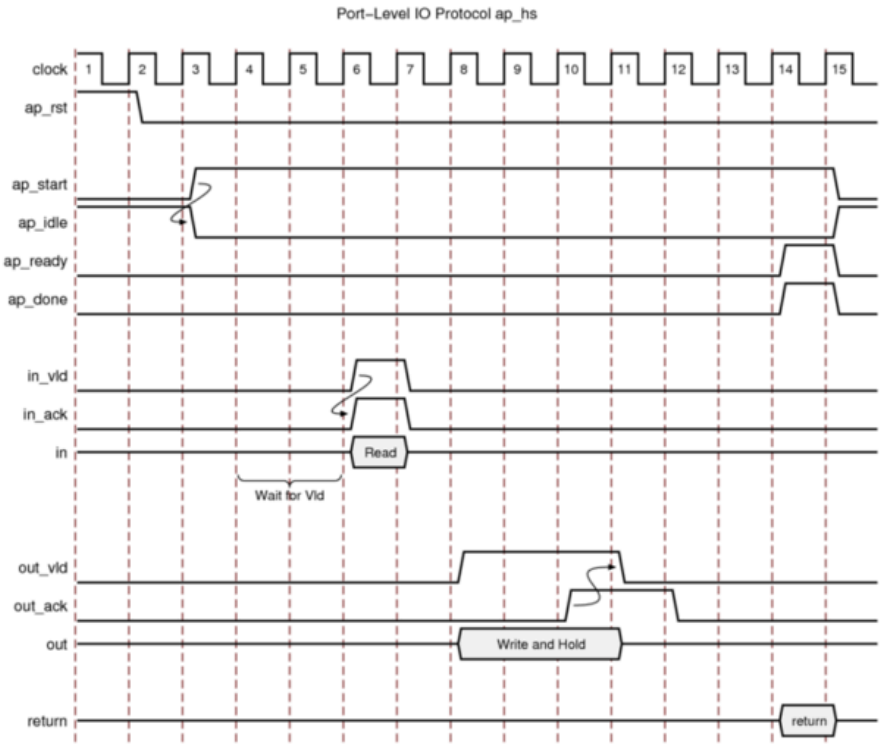
The ap_hs port-level I/O protocol provides the greatest flexibility in the development process, allowing both bottom-up and top-down design flows. Two-way handshakes safely perform all intra-block communication, and manual intervention or assumptions are not required for correct operation. The ap_hs port-level I/O protocol provides the following signals:

- Data port
- Valid signal to indicate when the data signal is valid and can be read
- Acknowledge signal to indicate when the data has been read

The following figure shows how an ap_hs interface behaves for both an input and output port. In this example, the input port is named in, and the output port is named out.

Note: The control signals names are based on the original port name. For example, the valid port for data input in is named in_vld.

Figure: Behavior of ap_hs Interface



For inputs, the following occurs:

- After start is applied, the block begins normal operation.
- If the design is ready for input data but the input valid is Low, the design stalls and waits for the input valid to be asserted to indicate a new input value is present.

  Note: The preceding figure shows this behavior. In this example, the design is ready to read data input in on clock cycle 4 and stalls waiting for the input valid before reading the data.
- When the input valid is asserted High, an output acknowledge is asserted High to indicate the data was read.

For outputs, the following occurs:

- After start is applied, the block begins normal operation.
- When an output port is written to, its associated output `valid` signal is simultaneously asserted to indicate valid data is present on the port.
- If the associated input acknowledge is Low, the design stalls and waits for the input acknowledge to be asserted.
- When the input acknowledge is asserted, indicating the data has been read, the output `valid` is deasserted on the next clock edge.

ap_ack

The `ap_ack` port-level I/O protocol is a subset of the `ap_hs` interface type. The `ap_ack` port-level I/O protocol provides the following signals:

- Data port
- Acknowledge signal to indicate when data is consumed
  - For input arguments, the design generates an output acknowledge that is active-High in the cycle the input is read.
  - For output arguments, Vitis HLS implements an input acknowledge port to confirm the output was read.

Note: After a write operation, the design stalls and waits until the input acknowledge is asserted High, which indicates the output was read by a consumer block. However, there is no associated output port to indicate when the data can be consumed.

⚠ CAUTION! You cannot use C/RTL co-simulation to verify designs that use `ap_ack` on an output port.

ap_vld

The `ap_vld` is a subset of the `ap_hs` interface type. The `ap_vld` port-level I/O protocol provides the following signals:

- Data port
- Valid signal to indicate when the data signal is valid and can be read
  - For input arguments, the design reads the data port as soon as the `valid` is active. Even if the design is not ready to read new data, the design samples the data port and holds the data internally until needed.
  - For output arguments, Vitis HLS implements an output `valid` port to indicate when the data on the output port is valid.

ap_ovld

The `ap_ovld` is a subset of the `ap_hs` interface type. The `ap_ovld` port-level I/O protocol provides the following signals:

- Data port
- Valid signal to indicate when the data signal is valid and can be read
  - For input arguments and the input half of inout arguments, the design defaults to type `ap_none`.
  - For output arguments and the output half of inout arguments, the design implements type `ap_vld`.

Port-Level I/O: Memory Interface Protocol

Array arguments are implemented by default as an `ap_memory` interface using word-addressing. This is a standard block RAM interface with data, address, chip-enable, and write-enable ports.
An `ap_memory` interface can be implemented as a single-port of dual-port interface. If Vitis HLS can determine that using a dual-port interface will reduce the initial interval, it will automatically implement a dual-port interface. The BIND_STORAGE pragma or directive can be used to specify the memory resource and if this directive is specified on the array with a single-port block RAM, a single-port interface will be implemented. Conversely, if a dual-port interface is specified using the BIND_STORAGE pragma and Vitis HLS determines this interface provides no benefit it will automatically implement a single-port interface.

If the array is accessed in a sequential manner an `ap_fifo` interface can be used. As with the `ap_hs` interface, Vitis HLS will halt if it determines the data access is not sequential, report a warning if it cannot determine if the access is sequential or issue no message if it determines the access is sequential. The `ap_fifo` interface can only be used for reading or writing, not both.

ap_memory

The `ap_memory` interface port-level I/O protocol implements arrays for arguments. This type of port-level I/O protocol connects to memory elements (for example, RAMs and ROMs) when the implementation requires random accesses to the memory address locations.

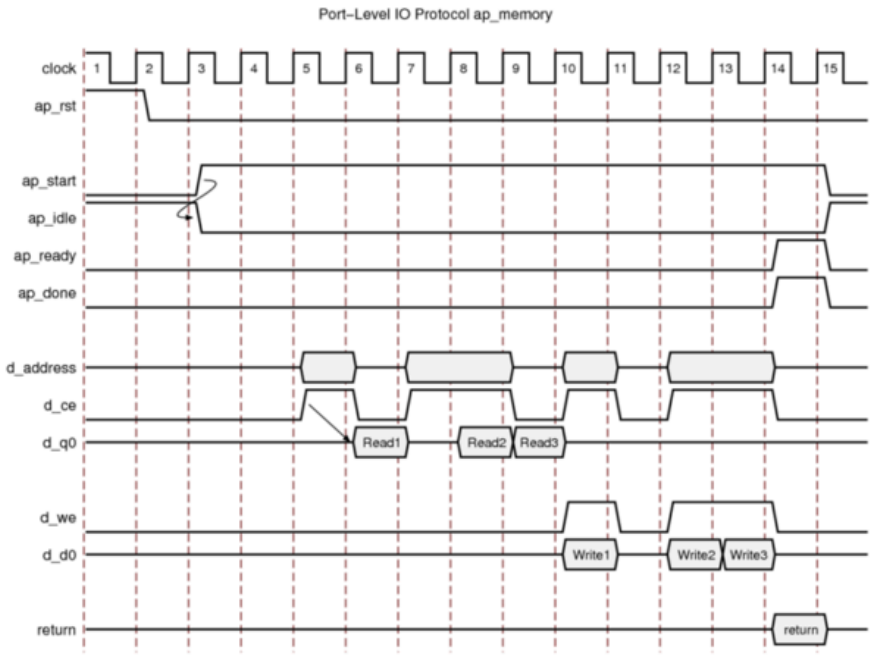
Note: If you only need sequential access to the memory element, use the `ap_fifo` interface instead. The `ap_fifo` interface reduces the hardware overhead.

The `ap_memory` interface uses word-based addressing, and generates a chip enabled control signal.
Since the v2024.1 release, the `ap_memory` interface appears as a single, grouped port. In IP integrator, you can use a single connection to create connections to all ports.
When using a memory interface, specify the implementation using the BIND_STORAGE pragma. If no target is specified for the arrays, Vitis HLS determines whether to use a single or dual-port RAM interface.

Tip: Before running synthesis, ensure array arguments are targeted to the correct memory type using the BIND_STORAGE pragma. Re-synthesizing with different memories can result in a different schedule and RTL.

The following figure shows an array named `d` specified as a single-port block RAM. The port names are based on the C/C++ function argument. For example, if the C/C++ argument is `d`, the chip-enable is `d_ce`, and the input data is `d_q0` based on the `output/q` port of the block RAM.

Figure: Behavior of ap_memory Interface



After reset, the following occurs:

- After start is applied, the block begins normal operation.
- Reads are performed by applying an address on the output address ports while asserting the output signal `d_ce`.

  Note: For a default block RAM, the design expects the input data `d_q0` to be available in the next clock cycle. You can use the BIND_STORAGE pragma to indicate the RAM has a longer read latency.

  Note: The external memory must stall and keep the output data valid when `d_ce` is low.
- Write operations are performed by asserting output ports `d_ce` and `d_we` while simultaneously applying the address and output data `d_d0`.

ap_fifo

When an output port is written to, its associated output `valid` signal interface is the most hardware-efficient approach when the design requires access to a memory element and the access is always performed in a sequential manner, that is, no random access is required. The `ap_fifo` port-level I/O protocol supports the following:

- Allows the port to be connected to a FIFO
- Enables complete, two-way `empty-full` communication
- Works for arrays, pointers, and pass-by-reference argument types

---

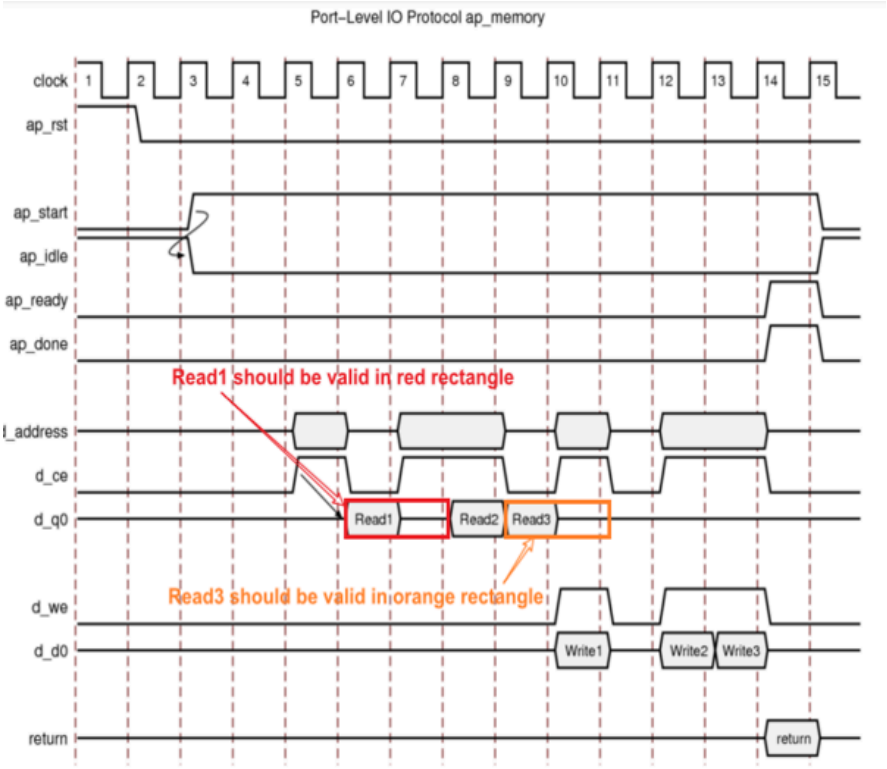
Note: Functions that can use an `ap_fifo` interface often use pointers and might access the same variable multiple times. To understand the importance of the `volatile` qualifier when using this coding style, see Multi-Access Pointers on the Interface.

In the following example, `in1` is a pointer that accesses the current address, then two addresses above the current address, and finally one address below.

```
void foo(int* in1, ...) {
 int data1, data2, data3;
       ...
 data1= *in1;
 data2= *(in1+2);
 data3= *(in1-1);
 ...
}
```

If `in1` is specified as an `ap_fifo` interface, Vitis HLS checks the accesses, determines the accesses are not in sequential order, issues an error, and halts. To read from non-sequential address locations, use the `ap_memory` interface.
You cannot specify an `ap_fifo` interface on an argument that is both read from and written to. You can only specify an `ap_fifo` interface on an input or an output argument. A design with input argument `in` and output argument `out` specified as `ap_fifo` interfaces behaves as shown in the following figure.

Figure: Behavior of ap_memory Interface with Latency 1



For inputs, the following occurs:

- After ap_start is applied, the block begins normal operation.
- If the input port is ready to be read but the FIFO is empty as indicated by input port `in_empty_n` Low, the design stalls and waits for data to become available.
- When the FIFO contains data as indicated by input port `in_empty_n` High, an output acknowledge `in_read` is asserted High to indicate the data was read in this cycle.

For outputs, the following occurs:

- After start is applied, the block begins normal operation.
- If an output port is ready to be written to but the FIFO is full as indicated by `out_full_n` Low, the data is placed on the output port but the design stalls and waits for the space to become available in the FIFO.
- When space becomes available in the FIFO as indicated by `out_full_n` High, the output acknowledge signal `out_write` is asserted to indicate the output data is `valid`.
- If the top-level function or the top-level loop is pipelined using the `-rewind` option, Vitis HLS creates an additional output port with the suffix `_lwr`. When the last write to the FIFO interface completes, the `_lwr` port goes active-High.

Programming Model for Multi-Port Access in HBM

High Bandwidth Memory (or HBM) provides high bandwidth by splitting arrays into different banks/pseudo-channels in the design. This is a common practice in partitioning an array into different memory regions in high-performance computing. The software application interacting with the hardware allocates a single buffer, which will be spread across the pseudo-channels.

Enabling Dependence Analysis on Different Arguments

Vitis HLS considers different pointers to be independent channels, and removes any dependency analysis. But the software application allocates a single buffer for both pointers, and this lets the tool maintain dependency analysis through `pragma HLS ALIAS`. The ALIAS pragma informs data dependence analysis about the pointer distance. Refer to the ALIAS pragma or directive for more information.
In the following code example, the kernel `arg0` is allocated in `bank0` and kernel `arg1` is allocated in `bank1`. The pointer distance should be specified in the `distance` option of the ALIAS pragma as shown below:

```
//Assume that the host code looks like this:
int *buf = clCreateBuffer(ctxt, CL_MEM_READ_ONLY, 2*bank_size, ...);
clSetKernelArg(kernel, 0, 0x20000000, buf);            // bank0
clSetKernelArg(kernel, 1, 0x20000000, buf+bank_size); // bank1

//The ALIAS pragma informs data dependence analysis about the pointer distance
void kernel(int *bank0, int *bank1, ...)
{
#pragma HLS alias ports=bank0,bank1 distance=bank_size
```

The ALIAS pragma can be specified using one of the following forms:

- Constant distance:

  ```
  #pragma HLS alias ports=arr0,arr1,arr2,arr3 distance=1024
  ```

- Variable distance:

  ```
  #pragma HLS alias ports=arr0,arr1,arr2,arr3 offset=0,512,1024,2048
  ```

Constraints:

- The depths of all the ports in the interface pragma must be the same
- All ports must be assigned to different bundles, bound to different HBM controllers
- The number of ports specified in the second form must be the same as the number of offsets specified, one offset per port. `#pragma HLS interface offset=off` is not supported
- Each port can only be used in one ALIAS pragma

Improving HBM Bandwidth with Multiple M_AXI Interfaces

Vitis HLS supports features on top-level array or pointer arguments to improve the parallelism and throughput achievable from C++ code with a few code changes.

Improvements to the ARRAY_PARTITION Pragma

The ARRAY_PARTITION pragma or directive can be applied to arrays on top-level arguments, at the interface of the top-level function. Used alongside the INTERFACE pragma, this combination lets Vitis HLS create multiple M_AXI interfaces for a single software argument to use with HBM Pseudo Channels. Use ARRAY_PARTITION `factor=N` to create N point-to-point connections between the M_AXI interfaces and HBM Pseudo Channels (or HBM_PC) to achieve up to N times the throughput of a single interface.
For the micro-architecture, an explicit parallelization of the array accesses and associated datapath is needed to match the increase of M_AXI interfaces. This translates to a similar parallelization from a coding perspective.
Array partitioning types can be complete, cyclic, or block, though for this application only cyclic or block are used. Each type can lead to a different micro-architecture.

- Cyclic partitions can be used within a loop: consecutive accesses of the array will connect to different M_AXI and their matching HBM_PC, and you can partially unroll the loop with the same factor.

```
constexpr int partitions = 4;
constexpr int NWORDS=32<<20;
void example(int a[NWORDS]  , int b[1]) {
    #pragma HLS INTERFACE m_axi port=a depth=NWORDS bundle=gmem max_widen_bitwidth=512
    #pragma HLS ARRAY_PARTITION cyclic variable=a factor=partitions
    #pragma HLS INTERFACE m_axi port=b depth=1 bundle=outmem
    int tot=0;
accessloop:
    for (int i=0; i < NWORDS; ++i) {
        #pragma HLS PIPELINE II=1
        #pragma HLS UNROLL factor=partitions
        tot+= a[i] * i; // computation on a[i]
    }
    b[0]=tot;
}
```

  The preceding example shows the use of a sized array rather than a pointer, and the depth option is provided with the INTERFACE pragma. Both methods are equivalent and at least one is needed for C-RTL co-simulation, to size the RTL buffers correctly. The sizing information on the arguments and interfaces is not needed when used with the Vitis target flow as pointers are sized to 64 bits automatically. Additionally, the cyclic partitioning is size-independent.
- Block partitions can be used when the design includes several instances of the same compute function, and each compute function will access a different block of the array via different M_AXI and their matching HBM_PC.

```
constexpr int partitions = 4;
constexpr int NWORDS=32<<20;
void my_pe(int a[NWORDS/partitions], int &b, int offset) { // PE processes a fraction of the
data
    // offset is needed to adjust the coefficient to multiply the array value
    int tot=0;
accessloop:
    for (int i=0; i < NWORDS/partitions; ++i) {
        #pragma HLS PIPELINE II=1
        tot += a[i] * (i+offset); // computation on a[i]
    }
    b=tot;
}

void reduce(int *partial_totals, int &out) {
     for (int i=tot=0; i < partitions; ++i) {
           #pragma HLS PIPELINE off
           tot+=partial_totals[i];
     }
     out = tot;
}

void example(int a[NWORDS ], int &b) {
     #pragma HLS INTERFACE m_axi port=a depth=NWORDS bundle=gmem
     max_widen_bitwidth=MAXWBW
     #pragma HLS ARRAY_PARTITION block variable=a factor=partitions
     #pragma HLS INTERFACE m_axi port=b depth=1 bundle=outmem
     int partial_totals[partitions], tot;
     #pragma HLS ARRAY_PARTITION complete variable=partial_totals
     #pragma HLS DATAFLOW
```

```
        instanceloop_unrolled:
        for (int i=0; i < partitions; ++i) {
        #pragma HLS UNROLL
              // each instance accesses a different block
              my_pe( &a[NWORDS*i/partitions], partial_totals[i], NWORDS*i/ partitions);
        }
        reduce(partial_totals, b);
    }
```

In the above example, the array sizing information is needed because the block partitioning is dependent on the size of the original array.
The M_AXI interfaces generated will have their bundle names derived from the original bundle name, with decimal suffix added. For example, pragma HLS INTERFACE bundle=gmem with a partition factor=4 will generate 4 M_AXI named m_axi_gmem_0, m_axi_gmem_1, m_axi_gmem_2 and m_axi_gmem_3. You will need to update the connectivity settings for the system to integrate all the M_AXI interfaces. Refer to *Mapping Kernel Ports to Memory* in *Embedded Design Development Using Vitis* (UG1701) for additional information.
In the examples above, the function signature used by the software application to call the `example()` module will change because of the use of `ARAY_PARTITION factor=4`. The function signature changes from `example(int a[NWORDS], int b[1])` to the following:

```
void example(
    int a_0[NWORDS/4],
    int a_1[NWORDS/4],
    int a_2[NWORDS/4],
    int a_3[NWORDS/4],
    int b[1]);
```

Generated Helper Functions for Host Side Array Partitioning

The software application will have to partition the buffer containing the array into the same cyclic or block partitioning used by the HLS design as described above. To help with this task, Vitis HLS generates helper functions to split host array buffers into multiple partitions, and the reverse helper functions to recombine several partitions into one array. Those helper functions are:

- `<top_name>_Set_<array_name>` is used to partition an array into several partitions. For example, with a `factor=4` and `int` datatype:

  ```
  void <DUT>_Set_<ARG>(int *dst[4], int *src, unsigned long long num_elements);
  ```

- `<top_name>_Get_<array_name>` is used to recombine several partitions into a single array. For example with a `factor=4` and `int` datatype:

  ```
  void <DUT>_Get_<ARG>(int *dst, int *src[4], unsigned long long num_elements)
  ```

  ★ **Tip:** The order of the arguments follows the convention: `destination(s)`, `source(s)`, `number of elements`.
- Generated helper functions are available as assembly files:
  ./project/solution/impl/ip/drivers/example_v1_0/src/hbm_helper_x86_64.s & hbm_helper_aarch64.s
- The generated helper functions are also available as compiled files:
  ./project/solution/impl/ip/drivers/example_v1_0/src/hbm_helper_x86_64.o & hbm_helper_aarch64.o

✎ **Note:** Header files are not generated automatically, so you need to include function declarations in the software application. The following example shows a `factor=4` and `int` datatype:

```
extern "C" void example_Set_a(int **, int *, long);
extern "C" void example_Get_a(int *, int **, long);
```

Example usage:

```
int *host_in=new int[NWORDS];
for (size_t i = 0; i < NWORDS; ++i) {
    host_in[i] = ... // initialize the host input memory
}

xrt::bo bo_Inputs[partitions];
int* bufIn_map[partitions];
for (int i = 0; i < partitions; i++) {
    // buffer object matching kernel arguments/memory banks
    bo_Inputs[i] = xrt::bo(device,NWORDS*sizeof(int)/partitions, krnl.group_id(i));
    // Map buffer object data to user pointer for manipulation
    bufIn_map[i] = bo_Inputs[i].template map<int*>();
}

// partition the input data into the mapped partitions
example_Set_a(bufIn_map, host_in, NWORDS);

xrt::run run(krnl); // run object for that kernel

for (int i = 0; i < partitions; i++) {
    // sync updated bo contents to board
    bo_Inputs[i].sync(XCL_BO_SYNC_BO_TO_DEVICE);
}
for (int i = 0; i < partitions; i++) {
    // set arguments according to function signature
    run.set_arg(i,bo_Inputs[i]);
}
...
run.start();
run.wait();
```

Managing Interfaces with SSI Technology Devices

Certain AMD devices use stacked silicon interconnect (SSI) technology. In these devices, the total available resources are divided over multiple super logic regions (SLRs). The connections between SLRs use super long line (SLL) routes. SLL routes incur delays costs that are typically greater than standard FPGA routing. To ensure designs operate at maximum performance, use the following guidelines:

- Register all signals that cross between SLRs at both the SLR output and SLR input.
- You do not need to register a signal if it enters or exits an SLR via an I/O buffer.
- Ensure that the logic created by Vitis HLS fits within a single SLR.

Note: When you select an SSI technology device as the target technology, the utilization report includes details on both the SLR usage and the total device usage.

If the logic is contained within a single SLR device, the HLS tool provides a `syn.rtl.register_all_io` configuration command. If the option is enabled, all inputs and outputs are registered. If disabled, none of the inputs or outputs are registered.

Vitis HLS Memory Layout Model

The Vitis application acceleration development flow provides a framework for developing and delivering FPGA accelerated applications using standard programming languages for both software and hardware components. The software component, or host program, is developed using C/C++ to run on x86 or embedded processors, with OpenCL/ Native XRT API calls to manage run time interactions with the accelerator. The hardware component, or kernel (that runs on the actual FPGA card/platform), can be developed using C/C++, OpenCL C, or RTL. The Vitis software platform promotes concurrent development and test of the hardware and software elements of a heterogeneous application. Due to this, the software program that runs on the host computer needs to communicate with the acceleration kernel that runs on the FPGA hardware model using well-defined interfaces and protocols. As a result, it becomes important to define the exact memory model that is used so that the data that is being read/written can be correctly processed. The memory model defines the way data is arranged and accessed in computer memory. It consists of two separate but related issues: data alignment and data structure padding. In addition, the Vitis HLS compiler supports the specification of special attributes (and pragmas) to change the default data alignment and data structure padding rules.

Data Alignment

Software programmers are conditioned to think of memory as a simple array of bytes and the basic data types are composed of one or more blocks of memory. However, the computer's processor does not read from and write to memory in single byte-sized chunks. Instead, today's modern CPUs access memory in 2, 4, 8, 16, or even 32-byte chunks at a time - although 32 bit and 64 bit instruction set architecture (ISA) architectures are the most common. Due to how the memory is organized in your system, the addresses of these chunks should be multiples of their sizes. If an address satisfies this requirement, then it is said to be *aligned*. The difference between how high-level programmers think of memory and how modern processors actually work with memory is pretty important in terms of application correctness and performance. For example, if you don't understand the address alignment issues in your software, the following situations are all possible:

- your software will run slower
- your application will lock up/hang
- your operating system can crash
- your software will silently fail, yielding incorrect results

The C++ language provides a set of fundamental types of various sizes. To make manipulating variables of these types fast, the generated object code will try to use CPU instructions that read/write the whole data type at once. This in turn means that the variables of these types should be placed in memory in a way that makes their addresses suitably aligned. As a result, besides size, each fundamental type has another property: its alignment requirement. It may seem that the fundamental type's alignment is the same as its size. This is not generally the case since the most suitable CPU instruction for a particular type may only be able to access a part of its data at a time. For example, a 32-bit x86 GNU/Linux machine may only be able to read at most 4 bytes at a time so a 64-bit `long long` type will have a size of 8 and an alignment of 4. The following table shows the size and alignment (in bytes) for the basic native data types in C/C++ for both 32-bit and 64-bit x86-64 GNU/Linux machines.

Table: Data Types

Type	32-bit x86 GNU/Linux		64-bit x86 GNU/Linux	
	Size	Alignment	Size	Alignment
bool	1	1	1	1
char	1	1	1	1
short int	2	2	2	2
int	4	4	4	4

Type	32-bit x86 GNU/Linux		64-bit x86 GNU/Linux	
	Size	Alignment	Size	Alignment
long int	4	4	8	8
long long int	8	4	8	8
float	4	4	4	4
double	8	4	8	8
long double	12	4	16	16
void*	4	4	8	8

Given the above arrangement, why does a programmer need to change the alignment? There are several reasons but the main reason will be to trade-off between memory requirements and performance. When you are sending data back and forth from the host computer and the accelerator, every byte that is transmitted has a cost. Fortunately, the C++ compiler provides the language extension `__attribute__ ((aligned(X)))` to change the default alignment for the variable, structures/classes, or a structure field, measured in bytes. For example, the following declaration causes the compiler to allocate the global variable x on a 16-byte boundary.

```
int x __attribute__ ((aligned (16))) = 0;
```

The `__attribute__((aligned (X)))` does not change the sizes of variables it is applied to, but may change the memory layout of structures by inserting padding between elements of the struct. As a result, the size of the structure will change. If you don't specify the alignment factor in an aligned attribute, the compiler automatically sets the alignment for the declared variable or field to the largest alignment used for any data type on the target machine you are compiling for. Doing this can often make copy operations more efficient because the compiler can use whatever instructions copy the biggest chunks of memory when performing copies to or from the variables or fields that you have aligned this way. The `aligned` attribute can only increase the alignment and can never decrease it. The C++ function offsetof can be used to determine the alignment of each member element in a structure.

Data Structure Padding

As shown in the table in Data Alignment, the native data types have a well-defined alignment structure but what about user-defined data types? The C++ compiler also needs to make sure that all the member variables in a struct or class are properly aligned. For this, the compiler inserts *padding* bytes between member variables. In addition, to make sure that each element in an array of a user-defined type is aligned, the compiler can add some extra padding after the last data member. Consider the following example:

Table: Alignment of Structs

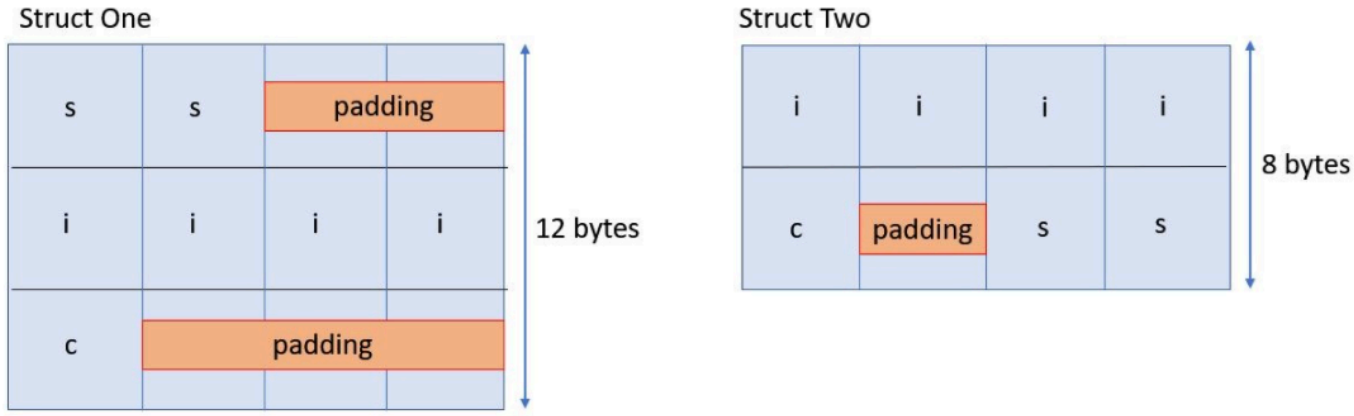
```
struct One
{
    short int s;
    int i;
    char c;
}
```

```
struct Two
{
   int i;
   char c;
   short int s;
}
```

The compiler always assumes that an instance of `struct One` starts at an address aligned to the most strict alignment requirement of all of the struct's members, which is `int` in this case. This is actually how the alignment requirements of user-defined types are calculated. Assuming the memory are on x86-64 alignment with `short int` having the alignment of 2 and `int` having an alignment of 4, to make the `i` data member of `struct One` suitably aligned, the compiler needs to

insert two extra bytes of padding between `s` and `i` to create alignment, as shown in the following figure. Similarly, to align data member `c`, the compiler needs to insert three bytes after `c`.
In the case of `struct One`, the compiler infers a total size of 12 bytes based on the arrangement of the elements of the struct. However, if the elements of the struct are reordered (as shown in `struct Two`), the compiler is now able to infer the smaller size of 8 bytes.

Figure: Padding of Structs



By default, the C/C++ compiler lays out members of a struct in the order in which they are declared, with possible padding bytes inserted between members, or after the last member, to ensure that each member is aligned properly. However, the C++ compiler provides a language extension, `__attribute__((packed))`which tells the compiler not to insert padding but rather allow the struct members to be misaligned. For example, if the system normally requires all `int` objects to have 4-byte alignment, the usage of `__attribute__((packed))` can cause `int` struct members to be allocated at odd offsets.
Usage of `__attribute__((packed))` must be carefully considered because accessing unaligned memory can cause the compiler to insert code to read the memory byte by byte instead of reading multiple chunks of memory at one time.

Vitis HLS Alignment Rules and Semantics

Given the behavior of the C++ compiler described previously, this section will detail how Vitis HLS uses `aligned` and `packed` attributes to create efficient hardware. First, you need to understand the Aggregate and Disaggregate features in Vitis HLS. Structures or class objects in the code, for instance internal and global variables, are disaggregated by default. Disaggregation implies that the structure/class is decomposed into separate objects, one for each struct/class member. The number and type of elements created are determined by the contents of the struct itself. Arrays of structs are implemented as multiple arrays, with a separate array for each member of the struct.
However, structs used as arguments to the top-level function are kept aggregated by default. Aggregation implies that all the elements of a struct are collected into a single wide vector. This allows all members of the struct to be read and written simultaneously. The member elements of the struct are placed into the vector in the order in which they appear in the C/C++ code: the first element of the struct is aligned on the LSB of the vector and the final element of the struct is aligned with the MSB of the vector. Any arrays in the struct are partitioned into individual array elements and placed in the vector from lowest to the highest order.

Table: Interface Arguments and Internal Variables

	Behavior without `AGGREGATE` pragma		Behavior with `AGGREGATE` pragma (compact=aut	
	Interface Argument	Internal Variable	Interface Argument	Internal variable
AXI protocol interface (`m_axi/s_axilite/axis`)	aggregate compact=**none**	N/A	compact=**none**	N/A

	Behavior without AGGREGATE pragma		Behavior with AGGREGATE pragma (compact=aut	
	Interface Argument	Internal Variable	Interface Argument	Internal variable
Struct/Class containing hls::stream object	**Automatically disaggregate the struct/class**	**Automatically disaggregate the struct/class**	N/A	N/A
other interface protocols	aggregate compact=**bit**	**Automatically disaggregated**	compact=**bit**	compact=**bit**

The goal of the default aggregation behavior in Vitis HLS is to use an x86_64-gnu-linux memory layout at the top level hardware interface while optimizing the internal hardware for better quality of results (QoR). The above table shows the default behavior of Vitis HLS. Two modes are shown in the table: the default mode where the AGGREGATE pragma is not specified by the user, and the case where the AGGREGATE pragma is specified by the user.
In the case of AXI4 interfaces (m_axi/s_axilite/axis), a structure is padded by default according to the order of elements of the struct as explained in Data Structure Padding. This aggregates the structure to a size that is the closest power of 2, and so some padding can be applied in this case. This in effect infers the compact=none option on the AGGREGATE pragma.
In the case of other interface protocols, the struct is packed at the bit-level, so the aggregated vector is only the size of the various elements of the struct, This in effect infers the compact=bit option on the AGGREGATE pragma.
The only exception to the above rules is when using hls::stream in the interface indirectly (i.e. the hls::stream object is specified inside a struct/class that is then used as the type of an interface port). The struct containing the hls::stream object is always disaggregated into its individual member elements.

Examples of Aggregation

Aggregate Memory Mapped Interface

This is an example of the AGGREGATE pragma or directive for an m_axi interface. The following is the aggregation_of_m_axi_ports example available on GitHub.

```
struct A {
  char foo;     // 1 byte
  short bar;    // 2 bytes
};

int dut(A* arr) {
#pragma HLS interface m_axi port=arr depth=10
#pragma HLS aggregate variable=arr compact=auto
  int sum = 0;
  for (unsigned i=0; i<10; i++) {
    auto tmp = arr[i];
    sum += tmp.foo + tmp.bar;
  }
  return sum;
}
```

For the above example, the size of the m_axi interface port arr is 3 bytes (or 24 bits) but due to the AGGREGATE compact=auto pragma, the size of the port will be aligned to 4 bytes (or 32 bits) as this is the closest power of 2. Vitis HLS will issue the following message in the log file:

```
INFO: [HLS 214-241] Aggregating maxi variable 'arr' with compact=none mode in 32-bits
(example.cpp:19:0)
```

★ **Tip:** The message above is only issued if the `AGGREGATE` pragma is specified. But even without the pragma, the tool will automatically aggregate and pad the interface port `arr` to 4 bytes as the default behavior for an AXI interface port.

Aggregate Structs on the Interface

This is an example of the AGGREGATE pragma or directive for an `ap_fifo` interface. The following is the aggregation_of_struct example available on GitHub.

```
struct A {
  int myArr[3];      // 4 bytes per element (12 bytes total)
  ap_int<23> length; // 23 bits
};

int dut(A arr[N]) {
#pragma HLS interface ap_fifo port=arr
#pragma HLS aggregate variable=arr compact=auto
  int sum = 0;
  for (unsigned i=0; i<10; i++) {
    auto tmp = arr[i];
    sum += tmp.myArr[0] + tmp.myArr[1] + tmp.myArr[2] + tmp.length;
  }
  return sum;
}
```

For `ap_fifo` interface, the struct will packed at the bit-level with or without aggregate pragma.
In the above example, the `AGGREGATE` pragma will create a port of size 119 bits for port `arr`. The array `myArr` will take 12 bytes (or 96 bits) and the element `length` will take 23 bits for a total of 119 bits. Vitis HLS will issue the following message in the log file:

```
INFO: [HLS 214-241] Aggregating fifo (array-to-stream) variable 'arr' with compact=bit mode
        in 119-bits (example.cpp:19:0)
```

Aggregate Nested Struct Port

This is an example of the AGGREGATE pragma or directive in the Vivado IP flow. The following is the aggregation_of_nested_structs example available on GitHub.

```
#define N 8

struct T {
   int m;   // 4 bytes
   int n;   // 4 bytes
   bool o;  // 1 byte
};

struct S {
   int p;   // 4 bytes
   T q;     // 9 bytes
};
  void top(S a[N], S b[N], S c[N]) {
#pragma HLS interface bram port=c
#pragma HLS interface ap_memory port=a
#pragma HLS aggregate variable=a compact=byte
#pragma HLS aggregate variable=b compact=bit
```

```
#pragma HLS aggregate variable=c compact=byte
    for (int i=0; i<N; i++) {
        c[i].q.m = a[i].q.m + b[i].q.m;
        c[i].q.n = a[i].q.n - b[i].q.n;
        c[i].q.o = a[i].q.o || b[i].q.o;
        c[i].p = a[i].q.n;
    }
}
```

In the above example, the aggregation algorithm will create a port of size 104 bits for ports a, and c as the compact=byte option was specified in the aggregate pragma but the compact=bit default option is used for port b and its packed size will be 97 bits. The nested structures S and T are aggregated to encompass three 32 bit member variables (p, m, and n) and one bit/byte member variable (o).

★ **Tip:** This example uses the Vivado IP flow to illustrate the aggregation behavior. In the Vitis kernel flow, port b will be automatically inferred as an m_axi port and will not allow the compact=bit setting.

Vitis HLS will issue the following messages in the log file:

```
INFO: [HLS 214-241] Aggregating bram variable 'b' with compact=bit mode in 97-bits
(example.cpp:19:0)
INFO: [HLS 214-241] Aggregating bram variable 'a' with compact=byte mode in 104-bits
(example.cpp:19:0)
INFO: [HLS 214-241] Aggregating bram variable 'c' with compact=byte mode in 104-bits
(example.cpp:19:0)
```

Examples of Disaggregation

Disaggregate AXIS Interface

This is an example of the DISAGGREGATE pragma or directive for an axis interface. The following is the disaggregation_of_axis_port example available on GitHub.

Table: Disaggregated Struct on AXIS Interface

HLS Source Code	Synthesized IP Module

`#define N 10`
 	
`struct A {`
`  char c;`
`  int i;`
`};`
 	
`void dut(A in[N], A out[N]) {`
`#pragma HLS interface axis port=in`
`#pragma HLS interface axis port=out`
`#pragma HLS disaggregate variable=in`
`#pragma HLS disaggregate variable=out`
`  int sum = 0;`
`  for (unsigned i=0; i<N; i++) {`
`  out[i].c = in[i].c;`
`  out[i].i = in[i].i;`
`  }`
`}`
	`module dut (`
	`ap_local_block,`
	`ap_local_deadlock,`
	`ap_clk,`
	`ap_rst_n,`
	`ap_start,`
	`ap_done,`
	`ap_idle,`
	`ap_ready,`
	`in_c_TVALID,`
	`in_i_TVALID,`
	`out_c_TREADY,`
	`out_i_TREADY,`
	`in_c_TDATA,`
	`in_c_TREADY,`
	`in_i_TDATA,`
	`in_i_TREADY,`
	`out_c_TDATA,`
	`out_c_TVALID,`

HLS Source Code	Synthesized IP Module
	out_i_TDATA, out_i_TVALID);

In the above disaggregation example, the struct arguments `in` and `out` are mapped to AXIS interfaces, and then disaggregated. This results in Vitis HLS creating two AXI streams for each argument: `in_c`, `in_i`, `out_c` and `out_i`. Each member of the struct `A` becomes a separate stream.
The RTL interface of the generated module is shown on the right above where the member elements `c` and `i` are individual AXI stream ports, each with its own TVALID, TREADY and TDATA signals.
Vitis HLS will issue the following messages in the log file:

```
INFO: [HLS 214-210] Disaggregating variable 'in' (example.cpp:19:0)
INFO: [HLS 214-210] Disaggregating variable 'out' (example.cpp:19:0)
```

Disaggregate HLS::STREAM

This is an example of the DISAGGREGATE pragma or directive when used with the `hls::stream` type.

Table: Disaggregated Struct of HLS::STREAM

HLS Source Code	Synthesized IP Module

#define N 1024
 	
struct A {,
hls::stream<int> s_in;
long arr[N];
};
 	
long dut(struct A &d) {
long sum = 0;
while(!d.s_in.empty())
sum += d.s_in.read();
for (unsigned i=0; i<N; i++)
sum += d.arr[i];
return sum;
}
	module dut (
	ap_local_block,
	ap_local_deadlock,
	ap_clk,
	ap_rst,
	ap_start,
	ap_done,
	ap_idle,
	ap_ready,
	d_s_in_dout,
	d_s_in_empty_n,
	d_s_in_read,
	d_arr_ce0,
	d_arr_q0,
	ap_return
	);

Using an `hls::stream` object inside a structure that is used in the interface will cause the struct port to be automatically disaggregated by the Vitis HLS compiler. As shown in the above example, the generated RTL interface will contain separate RTL ports for the `hls::stream` object `s_in` (named `d_s_in_*`) and separate RTL ports for the array `arr` (named `d_arr_*`).
Vitis HLS will issue the following messages in the log file:

```
INFO: [HLS 214-210] Disaggregating variable 'd'
INFO: [HLS 214-241] Aggregating fifo (hls::stream) variable 'd_s_in' with compact=bit mode in 32-
bits
```

Impact of Struct Size on Pipelining

The size of a struct used in a function interface can adversely impact pipelining of loops in that function that have access to the interface in the loop body. Consider the following code example which has two M_AXI interfaces:

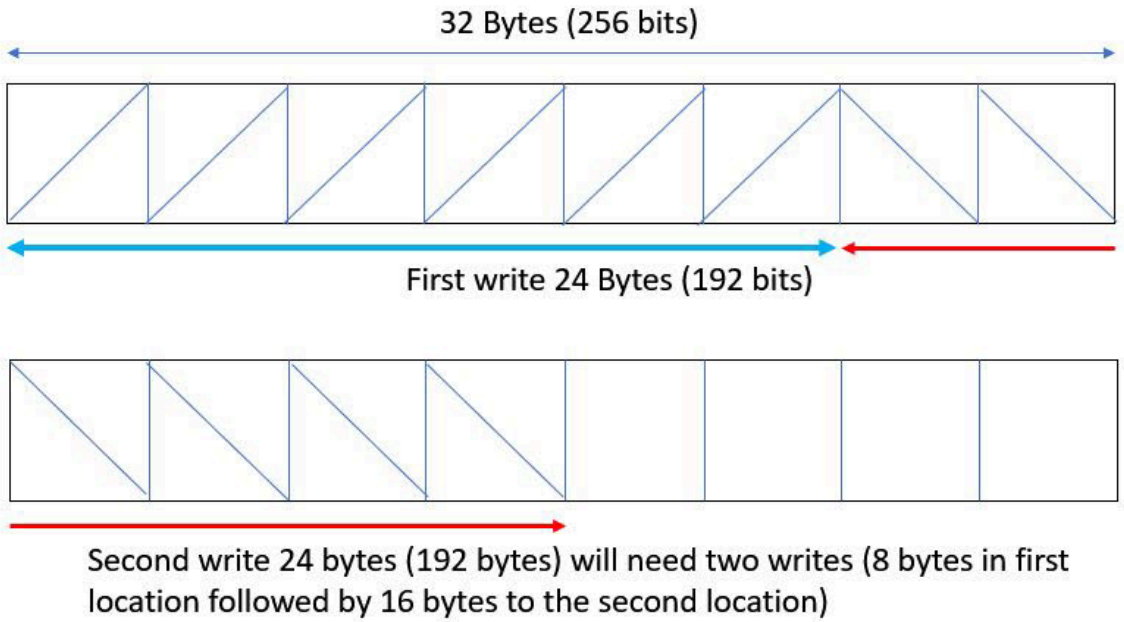
```
struct A { /* Total size = 192 bits (32 x 6) or 24 bytes */
    int s_1;
    int s_2;
    int s_3;
    int s_4;
    int s_5;
    int s_6;
};

void read(A *a_in, A buf_out[NUM]) {
READ:
    for (int i = 0; i < NUM; i++)
    {
        buf_out[i] = a_in[i];
    }
}

void compute(A buf_in[NUM], A buf_out[NUM], int size) {
COMPUTE:
    for (int j = 0; j < NUM; j++)
    {
        buf_out[j].s_1 = buf_in[j].s_1 + size;
        buf_out[j].s_2 = buf_in[j].s_2;
        buf_out[j].s_3 = buf_in[j].s_3;
        buf_out[j].s_4 = buf_in[j].s_4;
        buf_out[j].s_5 = buf_in[j].s_5;
        buf_out[j].s_6 = buf_in[j].s_6 % 2;
    }
}

void write(A buf_in[NUM], A *a_out) {
    WRITE:
    for (int k = 0; k < NUM; k++)
    {
        a_out[k] = buf_in[k];
    }
}

void dut(A *a_in, A *a_out, int size)
{
#pragma HLS INTERFACE m_axi port=a_in bundle=gmem0
#pragma HLS INTERFACE m_axi port=a_out bundle=gmem1
    A buffer_in[NUM];
    A buffer_out[NUM];

#pragma HLS dataflow
    read(a_in, buffer_in);
    compute(buffer_in, buffer_out, size);
    write(buffer_out, a_out);
}
```

In the above example, the size of struct `A` is 192 bits, which is not a power of 2. As stated earlier in the document, all AXI4 interfaces are by default sized to a power of 2. Vitis HLS will automatically size the two M_AXI interfaces (`a_in` and `a_out`) to be of size 256 - the closest power of 2 to the size of 192 bits (and report in the log file as shown below).

```
INFO: [HLS 214-241] Aggregating maxi variable 'a_out' with compact=none mode in
256-bits (example.cpp:49:0)
INFO: [HLS 214-241] Aggregating maxi variable 'a_in' with compact=none mode in 256-bits
(example.cpp:49:0)
```

This will imply that when writing the struct data out, the first write will write 24 bytes to the first buffer in one cycle but the second write will have to write 8 bytes to the remaining 8 bytes in the first buffer and then write 16 bytes into a second buffer resulting in two writes - as shown in the figure below.

Figure: Misaligned Write Cycles



This will cause the II of the WRITE loop in function `write()` to have an II violation since it needs II=2 instead of II=1. Similar behavior will happen when reading and therefore the `read()` function will also have an II violation since it needs II=2. Vitis HLS will issue the following warning for the II violation in function `read()` and `write()`:

```
WARNING: [HLS 200-880] The II Violation in module 'read_r' (loop 'READ'): Unable
to enforce a carried dependence constraint (II = 1, distance = 1, offset = 1) between
bus read operation ('gmem0_addr_read_1', example.cpp:23) on port 'gmem0' (example.cpp:23)
and bus read operation ('gmem0_addr_read', example.cpp:23) on port 'gmem0' (example.cpp:23).

WARNING: [HLS 200-880] The II Violation in module 'write_Pipeline_WRITE' (loop 'WRITE'):
Unable to enforce a carried dependence constraint (II = 1, distance = 1, offset = 1)
between bus write operation ('gmem1_addr_write_ln44', example.cpp:44) on port 'gmem1'
(example.cpp:44) and bus write operation ('gmem1_addr_write_ln44', example.cpp:44) on
port 'gmem1' (example.cpp:44).
```

The way to fix such II issues is to pad struct `A` with 8 additional bytes such that you are always writing 256 bits (32 bytes) at a time or by using the other alternatives shown in the table below. This will allow the scheduler to schedule the reads/writes in the READ/WRITE loop with II=1.

Table: Struct Alignment

Code Block	Description

`struct A {`
`  int s_1;`
`  int s_2;`
`  int s_3;`
`  int s_4;`
`  int s_5;`
`  int s_6;`
`  int pad_1;`
`  int pad_2;`
`};`
	Defines the total size of the struct as 256 bits (32 x 8) or 32 bytes, by adding required padding elements.

`struct A {`
`  int s_1;`
`  int s_2;`
`  int s_3;`
`  int s_4;`
`  int s_5;`
`  int s_6;`
`} __attribute__ ((aligned(32)));`
	Uses the standard `__aligned__` attribute.

`struct alignas(32) A {`
`  int s_1;`
`  int s_2;`
`  int s_3;`
`  int s_4;`
`  int s_5;`
`  int s_6;`
`}`
	Uses the C++ standard `alignas` type specifier to specify custom alignment of variables and user defined types.

Execution Modes of HLS Designs

The execution mode of the HLS design refers to the way the design works as a block (or module) both with regard to itself and the functions within the block, or in relationship with other blocks modules, or outside software that addresses the block. These modes are determined by block control protocols assigned to the HLS design as described in Block-Level Control Protocols, and by the internal structure of the HLS design as described in Abstract Parallel Programming Model for HLS.
Execution modes of kernels include:

Overlapping Mode

Let the next execution of a new transaction begin before the current transaction is complete. Function pipelined, loop rewind or dataflow execution allows overlapping block runs to begin processing additional data as soon as the design is ready.

Control Protocols: Use the `ap_ctrl_chain` or `ap_ctrl_hs` with backpressure support. Backpressure is a mechanism that allows downstream modules to control the flow of data. If a downstream module is not ready to receive new data, it can assert a signal to stall the upstream modules temporarily. This prevents data loss and ensures synchronization across different stages of the pipeline.

Sequential Mode

Requires the current transaction to complete before a new transaction can be started.

Control Protocols: Can be implemented using `ap_ctrl_hs` or `ap_ctrl_chain` without backpressure support. By eliminating backpressure, the design assumes that data is consumed at a consistent rate, and there is no need for flow control between stages.

Continuously Running Mode

Continuously running kernels run indefinitely until explicitly reset or run for a certain number of times, making them fundamentally different from classical software acceleration kernels. In contrast to traditional software kernels that are called, execute, and return upon completion, continuously running kernel continuously executes, mimicking a `while(1)` loop. These kernels are especially useful in hardware contexts where data streams in without predefined boundaries or known completion points.

Continuously running kernel are commonly used in applications where data is continuously fed into the system, and the kernel must process it without interruption. Examples include:

- **Networking**: Handling incoming data packets from Ethernet ports without the host system knowing the total number of packets in advance. For instance, a kernel might process packets from `TVALID` (valid data) to `TLAST` (last packet), with continuous execution.
- **Firewall**: A simple rule-based firewall where the rules are written once by the host at reset, and the kernel processes packets continuously. The host can read packet drop counters at any time, but these counters are the result of a single kernel execution.
- **Network Routing**: A network router that updates its routing table during execution but doesn't stop to process incoming packets continuously.
- **Load Balancing**: A load balancer that routes data based on a hash map, updating server lists and IP addresses while continuously handling traffic.
- **Fintech**: Financial applications where data packets from Ethernet ports are processed continuously, and the host loosely monitors the execution status without interrupting the ongoing operations.

Control Protocols: Continuously Running mode can be modeled using both the scenarios.

- Control Driven Task-level Parallelism (CDTLP) in auto-restart mode using `ap_ctrl_chain` block level protocol.
- Data-driven Task-level Parallelism (DTLP) using `ap_ctrl_none` block level protocol.

Auto-Restarting

Auto-restart mode lets the HLS design automatically restart the module when it is ready to begin processing additional data. This mode supports both overlapping and sequential execution. Auto-restarting is the approach for data-driven TLP designs, but can also be implemented in control-driven TLP designs as described in Auto-Restarting Mode.

Block-Level Control Protocols

The execution mode of a Vitis kernel or Vivado IP is defined by the block-level control protocol and the structure of sub-functions within the HLS design. For control-driven TLP the `ap_ctrl_chain` and `ap_ctrl_hs` protocols support both sequential or pipelined execution. For data-driven TLP `ap_ctrl_none` is the required control protocol.

The `ap_ctrl_chain` control protocol is the default for the Vitis kernel flow as explained in Interfaces for Vitis Kernel Flow. The `ap_ctrl_hs` block-level control protocol is the default for the Vivado IP flow as described in Interfaces for Vivado IP Flow. However, you should use `ap_ctrl_chain` when chaining HLS designs together to better support pipelined execution.

You can specify the block-level control protocol on the function return using the INTERFACE pragma or directive. If the C/C++ code does not return a value, you can still specify the control protocol on the function return. If the C/C++ code uses a function return, Vitis HLS creates an output port `ap_return` for the return value.

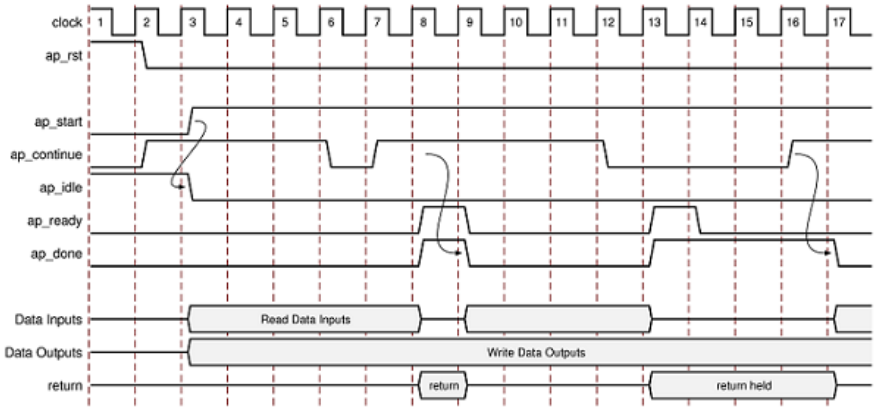
★ **Tip:** When the function return is specified as an AXI4-Lite interface (`s_axilite`) all the ports in the control protocol are bundled into the `s_axilite` interface. This is a common practice for software-controllable kernels or IP when an application or software driver is used to configure and control when the block starts and stops operation. This is a requirement of XRT and the Vitis kernel flow.

---

ap_ctrl_chain

The following figure shows the behavior of the block-level handshake signals created by the `ap_ctrl_chain` control protocol for a sequential execution. In the following figure, the first transaction of the HLS design completes, and the second transaction starts immediately because `ap_continue` is High when `ap_done` is High. However, the design halts at the end of the second transaction until `ap_continue` is asserted High.

Figure: Behavior of ap_ctrl_chain Interface



The timing diagram displays the following behavior after reset occurs:

1. The block waits for `ap_start` to go High before it begins operation.
2. Output `ap_idle` goes Low immediately to indicate the design is no longer idle.
3. The `ap_start` signal must remain High until `ap_ready` goes High. Once `ap_ready` goes High:
    - If `ap_start` remains High the design will start the next transaction.
    - If `ap_start` is taken Low, the design will complete the current transaction and halt operation.
4. Data can be read on the input ports.
5. Data can be written to the output ports.

    Note: The input and output ports can also specify a port-level I/O protocol that is independent of the control protocol. For details, see Port-Level Protocols for Vivado IP Flow.
6. Output `ap_done` goes High when the block completes operation.

    Note: If there is an `ap_return` port, the data on this port is valid when `ap_done` is High. Therefore, the `ap_done` signal also indicates when the data on output `ap_return` is valid.
7. The `ap_ctrl_chain` control protocol provides an active-High `ap_continue` signal that indicates when the downstream block that consumes the output data is ready for new data inputs. This allows the downstream block to provide back-pressure to prevent the flow of data.
    - If the `ap_continue` signal is High when `ap_done` is High, the design continues operating.
    - If the downstream block is not able to consume new data inputs, the `ap_continue` signal is Low. If the `ap_continue` signal is Low when `ap_done` is High, the design stops operating, the `ap_done` signal remains High waiting for `ap_continue` to go High.
8. When the design is ready to accept new inputs, the `ap_ready` signal is pulsed High for one clock cycle. The `ap_ready` port of a downstream block can directly drive the `ap_continue` port. Following is additional information about the `ap_ready` signal:
    - The `ap_ready` signal is inactive until the design starts operation.
    - In non-pipelined designs, the `ap_ready` signal is asserted at the same time as `ap_done`.
    - In pipelined designs, the `ap_ready` signal might go High at any cycle after `ap_start` is sampled High. This depends on how the design is pipelined.
    - If `ap_start` remains high after `ap_ready` goes high, the next transaction starts immediately.
    - When `ap_start` goes low right after `ap_ready` goes high, the design keeps executing until `ap_done` is high and then stops operation, unless `ap_start` goes high again in the meantime, which starts a new transaction.

9. The ap_idle signal indicates when the design is idle and not operating. Following is additional information about the ap_idle signal:
    - If the ap_start signal is Low when ap_ready is High, the design stops operation, and the ap_idle signal goes High one cycle after ap_done.
    - If the ap_start signal is High when ap_ready is High, the design continues to operate, and the ap_idle signal remains Low.

ap_ctrl_hs

The ap_ctrl_hs control protocol has the same signals as ap_ctrl_chain, but sets the ap_continue signal to 1 so it remains high. This control protocol supports sequential and pipelined execution modes, but does not offer back-pressure from downstream design modules to control the flow of data.

ap_ctrl_none

ap_ctrl_none also has the same signals as ap_ctrl_chain, but the handshake signal ports (ap_start, ap_idle, ap_ready, and ap_done) are set high and optimized away.
An ap_ctrl_none region, or one or more hls::tasks, can be instantiated in one of two ways:

- Either with only ap_ctrl_none regions all the way to the top, including the top (i.e. no sequential FSM or non-ap_ctrl_none dataflow anywhere above, even due to a coding mistake),
- Or in a dataflow region where:
    - its input streams are produced by one or more non-ap_ctrl_none processes or regions that are called before it
    - its output streams are consumed by one or more non-ap_ctrl_none processes or regions that are called after it

The former enables the elimination of the ap_start/ap_ready/ap_done/ap_continue handshakes all the way to the top. The latter enables the generation of the ap_start/ap_ready handshake for the dataflow region above by the predecessor processes or regions, and of the ap_done/ap_continue handshake by the successor processes or regions.

!! Important: If you use the ap_ctrl_none control protocol in your design, you must meet at least one of the conditions for C/RTL co-simulation as described in Co-simulation Requirements for Interface Synthesis to verify the RTL design. If at least one of these conditions is not met, C/RTL co-simulation halts with the following message:

```
@E [SIM-345] Cosim only supports the following 'ap_ctrl_none' designs: (1)
combinational designs; (2) pipelined design with task interval of 1; (3) designs with
array streaming or hls_stream ports.
@E [SIM-4] *** C/RTL co-simulation finished: FAIL ***
```

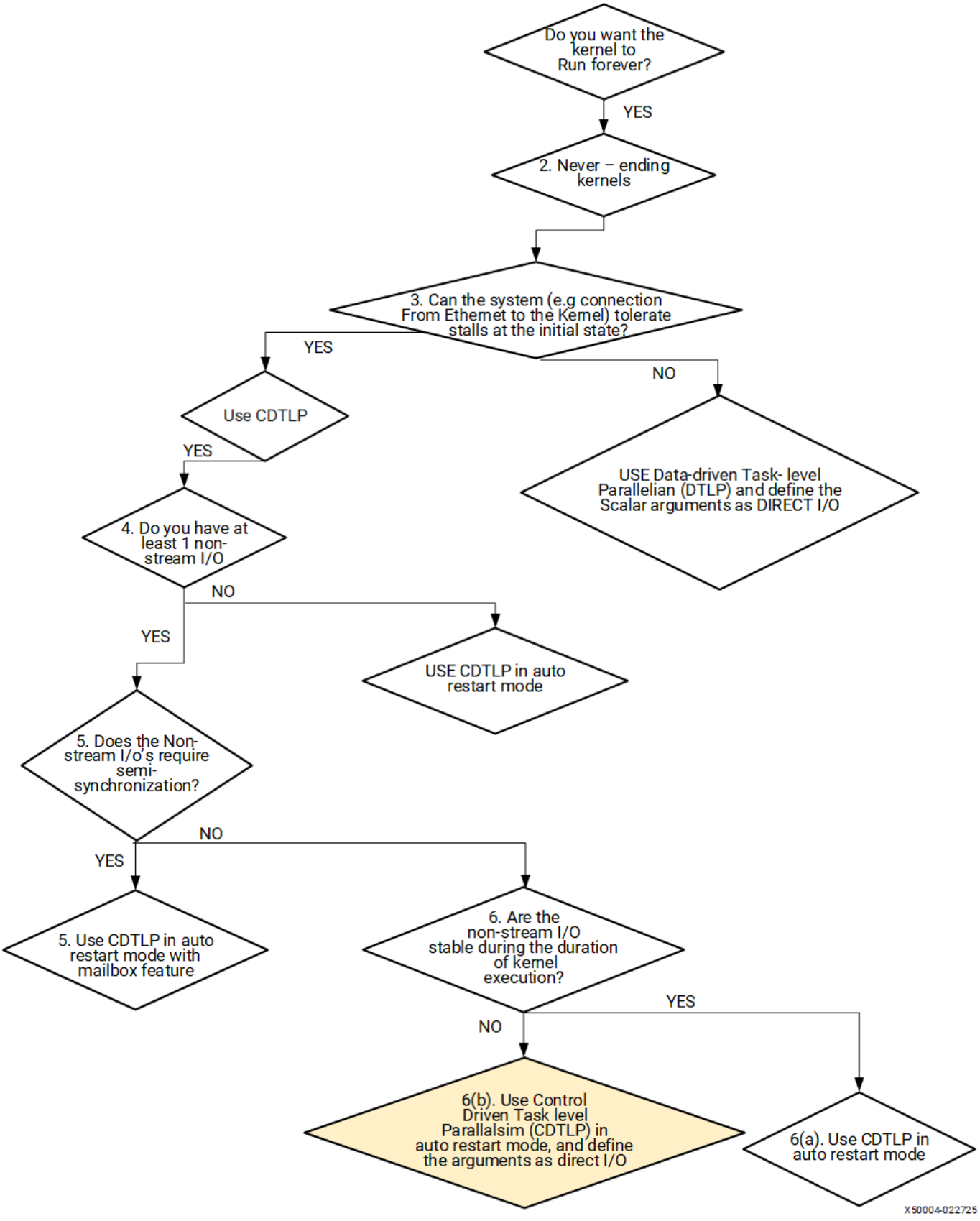
Continuously Running Mode

The continuously running kernels are kernels that will run forever, until they are reset. The notion of a never-ending kernel is typical of an HW design context. These kernels are closely related to HW FSMs, modeled as processes in VHDL or Verilog. They are very different from classical SW acceleration kernels, which are meant to "be called" and "return" when done in a manner akin to function calls in SW. The behavior of a never-ending kernel differs from that of an SW acceleration kernel. Consider the below example; as a classical SW acceleration kernel, it is meant to "be called" and "return" when done. But a never-ending kernel will re-execute until reset without explicitly being started by the host; it is similar to a while (1) loop around the function.
The use case is in the field of video/networking/fintech where the kernel receives data continuously without any interrupt from the Ethernet ports/DAC etc. This use case needs Continuously running kernels because the host does not know in advance the number of packets (where the number of packets is counted from the first TVALID packet to the last TLAST

packet) the kernel should process. This problem is solved when the kernel is modeled with a Continuously running mode, allowing the kernel to run forever without knowing the number of packets in advance.
Some other examples are:

- Simple rule-based “firewall”, with “rules” written by the host code at reset time, and where the host code can read a set of counters of dropped packets at any time, but where all counter values must come from a single kernel execution.
- Network router where the routing table must be updated entirely for kernel execution.
- A load balancer that uses a hash map to send data to a server must update the server list, server map, and corresponding IP addresses simultaneously.
- Fintech Example where the host loosely monitors the execution status of the kernel running on the accelerator card processing requests coming in over the Ethernet ports.

Figure: Kernel Flow Chart



1. **Can the system tolerate a stall at the initial state**

   There are two ways to model the Continuously running kernel, but the decision of option-1/2 will depend on the system designer.

   - **Option 1:** Data-driven Task-level Parallelism (DTLP).
   - **Option 2:** Control and Data-Driven Task-level Parallelism (CDTLP) in auto restart mode.

   Option 1: In Data-driven Task-level Parallelism, the kernel will start as soon as the `xclbin` is loaded onto the device; there won't be any overhead for starting the kernel. This choice is best suited for designs that **cannot**

tolerate or prefer not to for any stalls on the output side of Ethernet/DAC.

Option 2: In Control and Driven Task-level Parallelism (CDTLP), the host must set the kernel in auto restart mode. This would involve some startup overhead compared to option -1, but the kernel designer needs to do this only once for the entirety of the application run. Similarly, the kernel designer is responsible for stopping the kernel when the application has finished.

In summary, the decision to use options 1 and 2 depends on **how the kernel starts and its pros and cons.** Once the modeling style is decided, the kernel designer must decide whether they have at least one non-stream I/O.

2. **What is a non-stream I/O**

   Vitis HLS supports memory, stream, and register paradigms, where each paradigm follows a certain interface protocol to interact with the external world. The different paradigms description is documented at Interfaces for Vitis Kernel Flow.

   - **Non-Stream I/O:**
     - Non-stream I/O falls under the following Interface paradigms:
       - Memory Paradigm
       - Register Paradigm
   - **Stream I/O:**
     - Stream I/O falls under the following Interface paradigms:
       - Stream Paradigm

3. Need for Semi-synchronization for non-stream I/O

   Semi-synchronization exchanges I/O data to/from auto restart kernel in an asynchronous, non-blocking way. In an SW acceleration kernel, the I/O is fully synchronized. This means the kernel waits for all the inputs to be available before starting the kernel. In a Semi- synchronized I/O, from the second function start, the kernel will use the old values and does not block the kernel execution for the new updated values. Semi-synchronized I/O can be used in video application that streams continuously. A continuously streamed design might require changing the requirements asynchronously, non-blocking. For example, In the middle of the application run, a video application might want to increase the output image's resolution from HD to 4K. Using the Semi-synchronization feature, the application can change the input parameters without blocking the kernel execution. In HLS, this feature is implemented as a *Mailbox* feature. Refer to Mailbox Semantics.

   Once the kernel design has decided on semi-synchronization, the kernel designer needs to be aware of the behavior of I/O ports.

4. The behavior of I/O ports: Stable/Direct I/O

   Depending on the behavior of the input(s), there are two cases when an input (non-stream) argument can change with respect to a kernel execution:

   - Case (a): The I/O never changes during the whole execution of the kernel.
   - Case (b): Can change at any time, regardless of whether the kernel is executing or idle.

   Case (a): The I/O never changes during the whole execution of the kernel.

   - The inputs that only change when the kernel is idle are, by default, treated as stable by the compiler. This means the compiler assumes these inputs will remain constant during the active execution of the kernel. In the example below, the argument "a" is sampled during the kernel's idle state and retains the same value for the entire duration of the kernel's execution.

     ```
     void func(hls::stream<int> in, int a,hls::stream<int>out)
     {
     #pragma HLS INTERFACE mode=ap_ctrl_chain
         do {
             out.write(in.read()+a);
          }
     }
     ```

   Case (b): Can change at any time, regardless of whether the kernel is executing or idle.

- Kernels whose arguments are modeled using hls::direct I/O class can change the value at any point in the kernel execution. The kernel designer is not particular about the timing of the change if the kernel can see the updated values at some point in future time. In the example below, the argument "reset_myCounter" is sampled whenever there is valid input during the kernel's execution.

```
void krnl_stream_vdatamover(hls::stream<pkt> &in,
                      hls::stream<pkt> &out,
                                int mem[DATA],
                      hls::ap_vld<int> &reset_value,
                      hls::ap_vld<int> &reset_myCounter
                      )


{for(int i=0;i<DATA;i++)
{
...
  if(reset_myCounter.valid())


    {
        int reset = reset_myCounter.read();
    }
..
}
}
```

Auto-Restarting Mode

Vitis HLS designs can be control-driven modules or data-driven modules, as described in Abstract Parallel Programming Model for HLS. This means that the execution of the HLS design is managed through control signals, and managed by an external software application, or by software drivers, or is driven by the presence of data at the inputs.
Data-driven modules are auto-restarting designs by the mere fact of additional data to be processed. The HLS design finishes execution of one transaction and begins execution of the next transaction when data is available. Execution stalls when no data is present, and restarts when data is present. This section is not concerned with these data-driven designs.
For control-driven modules though, the managing software application or a connected HLS design must trigger the `ap_start` signal of the module to begin processing, as described in Block-Level Control Protocols. While these designs are not auto-restarting by default, they can be implemented using auto-restarting mechanisms as described here. This auto-restarting mode is useful for control-drivel TLP when the HLS design uses streaming I/O for data input or output, but also has control signals allowing it to be managed by a software application. In this case the software application starts the kernel one time and the kernel runs for a specified number of transactions, or restarts continuously until it is reset or explicitly stopped by the software application.
This control-driven auto-restarting design is useful for designs with streaming data coming from and going to the I/O pins (Ethernet, SerDes) of the AMD device, or data streamed from or to connected HLS design modules. In other words, the HLS design is primarily data-driven, but not exclusively.

Working with Auto-Restarting Designs

Auto-restarting designs can run continuously after being started once by the software application. They can run continuously until reset and restarted, or they can be programmed to run for a predetermined number of iterations without the software application explicitly calling them multiple times in a mode called counted auto-restart. This functionality is similar to a `while(running)` loop in software code, where the `running` variable is controlled by the software application. The control of the design is managed in hardware so that once the block is started by the software application it is automatically restarted until the iteration count is exceeded, or until explicitly stopped by the host code. In addition, the application can query the status of the design to check specific register states or to provide new parameters to be used at the next opportunity.

Auto-restarting designs make use of several unique features:

1. The auto-restart bit in the control register is used to continuously restart the design, or restart it for a specified number of iterations, without explicit software calls for each execution.
2. A mailbox feature of the Xilinx Runtime (XRT) to enable the software application to occasionally synchronize with the design to set new operating parameters, or check the status of the current run, as described in Using the Mailbox.
3. The `software_reset` feature letting the software application reset the design to stop execution.

Supported Interfaces

Auto-restarting HLS design require the `ap_ctrl_chain` or `ap_ctrl_hs` protocol specified by Vitis HLS, where `ap_ctrl_chain` is the recommended protocol.

- The design should specify the `mode=ap_ctrl_chain` on the INTERFACE pragma or directive.
- Auto-restarting designs support streaming interfaces (`axis`) and both scalar arguments (`s_axilite`) and memory mapped (`m_axi`) arguments which can be both read and written.

Auto-restart designs can be used in a couple of different scenarios:

1. Auto-restart kernel with streaming interfaces (`axis`) only, that does not require the kernel to interact with the host application at all. Examples of this would be a Fast-Fourier Transform (FFT) with configuration data compiled into the kernel, or FIR filters with coefficients compiled into the kernel.
2. Auto-restart kernel using scalars and memory mapped (`m_axi`) arguments, either with `axis` interfaces or without. The scalar and memory mapped arguments require the mailbox to update the kernel parameters when needed. Examples of this would include a simple rule-based firewall with rules written by the host application at reset, with a counter of dropped packets that can be read by the host code but where all values must come from a single kernel execution. Or, a load balancer that uses a hash map to send data to a server, must update the server list, server map, and corresponding IP addresses simultaneously.

Enabling Auto-Restart

To enable a continuously restarting HLS design specify the following in a configuration file:

```
syn.interface.s_axilite_auto_restart_counter=1
```

To enable auto-restart with the mailbox features in the HLS design use the following configuration commands:

```
syn.interface.s_axilite_mailbox=both
syn.interface.s_axilite_auto_restart_counter=1
```

You can enable the software reset feature for auto-restart using the following option:

```
syn.interface.s_axilite_sw_reset=1
```

★ **Tip:** Resetting the kernel stops all the incoming traffic on the ports and flushes out any outstanding MAXI transactions. Refer to Using Auto-Restart with the Mailbox for more information on using reset.

Using the Mailbox

The main advantage of auto-restarting HLS designs is that they run semi-autonomously operating as data-driven without the need for frequent interaction with the software application or for software control. But auto-restarting kernels also offer semi-synchronization through a feature known as the mailbox, which provides the ability to exchange data with the software application in an asynchronous, non-blocking, and safe way.
Once started by the software application the HLS design is automatically restarted until explicitly stopped. The software application can also query the status of the design to determine when it actually has finished executing after being

instructed to do so. The application and auto-restarting design use the following communications protocol:

- For passing argument values from the software application to the design, the mailbox implements a set of double-buffered `s_axilite` mapped registers to ensure non-blocking communication and consistent passing of inputs by the software application and passing of outputs by the design.
- Whenever the software application writes to an input argument, it changes the software-side copy. The HLS design running in hardware does not see that change. After the software application requests a mailbox write, the next time the design automatically restarts the copy of the registers that are seen by the design is updated with the latest inputs. Hence the software application can write any number of arguments in any order, and the HLS design does not see these updates until the software application requests a mailbox write, and the design restarts.

  ★ **Tip:** If some arguments are arrays mapped to `s_axilite` register files, then the entire array must be written between successive mailbox writes because it is implemented as a ping-pong buffer.
- The same process occurs on the output side when the HLS design writes to the design-side register and requests a mailbox read. The next time the design is done, the values of the `s_axilite` mapped output arguments are updated, and the software application can read them as needed.

Hence the software application has the following criteria:

- Not in charge of providing input data at every execution of the HLS design and collecting output data at its finish, as it would be in the case of control-driven designs.
- Involved in occasionally setting and updating some design input parameters (for example, routing tables, etc.) and checking the status of the design execution. This semi-synchronization operation is typically done without a fixed communication rate between the software application and the HLS design.
- When the software application has a new set of parameters to send to the HLS design, it does so without caring where the design is in its execution. When the software application needs to check the status of the HLS design, it does so without caring where design is in its execution. It is satisfied only that the parameter update and status check is performed consistently for the hardware.

Mailbox Semantics

The mailbox features can be used for both input and output, and would need to be specified for all `s_axilite` I/Os of an HLS component. It is enabled with a global option for the interfaces using the following configuration commands:

```
syn.interface.s_axilite_mailbox=both
syn.interface.s_axilite_auto_restart_counter=1
```

After setting up the configuration options, the mailbox implements a pair of registers called `HW copy` and `SW copy`. The input mailbox and output mailbox has an independent pair of registers. Communication from the software application with the HLS component includes:

Input Mailbox

- The application writes some or all elements to the `SW copy` register of the mailbox.
- The application notifies the mailbox that `SW copy` is updated.
- When the HLS design restarts, the `SW copy` register is copied to the `HW copy` register.
- The application is notified that the `HW copy` has been updated, and can change the `SW copy` register again as needed.

  ★ **Tip:** Multiple reads by the HLS design can occur without an update from the software application.

Output Mailbox

- The application notifies the mailbox that it wants to read an updated copy of the mailbox. The HLS design writes some or all the elements to the `HW copy` register of the mailbox at the end of the current execution.
- When the design is done, `HW copy` is copied to the `SW copy` register.
- The application is notified that `SW copy` is updated and can read it at any time.

> ★ **Tip:** Multiple writes by the hardware can occur without any software request to update.

Using Auto-Restart in Cosimulation (C/RTL Cosim)

Auto-Restart enables a kernel/IP to re-launch itself immediately after each completion for a finite number of iterations, without software intervention or gaps. In co-simulation flow with AXI4-Lite control, the cosim framework create 4 cycle gaps between successive DUT launches, preventing a seamless no-gap dut execution
The Auto-Restart helper class eliminates startup gaps during co-simulation (cosim), ensuring continuous, gap-free kernel execution for the specified number of iterations.

Programmer's Model

When invoked, the AXI4-Lite (SAXI4-Lite) interface writes the iteration count(N) to the kernel's Auto-Restart counter register at address 0x10. The kernel then executes all iterations back-to-back with no gaps.

1. Include the header: Add #include "hls/task.h" to access the Auto-Restart helper class
2. Set up your test bench: Initialize variables and buffers with data for the DUT
3. Invoke with Auto-Restart: Call the DUT using hls::autorestart name_of_class_instance(num_iters, dut_function, arg1, arg2, ...); where N is the number of iterations

When invoked, the AXI4-Lite (SAXI4-Lite) interface writes the iteration count(N) to the kernel's Auto-Restart counter register at address 0x10. The kernel then executes all iterations back-to-back with no gaps.

Example

```
#include "hls_task.h"

void main()
{
 for(i.. )
 {
  test[i] = i;
 }
 // Run 3 iterations back-to-back with test
 hls::autorestart run_top(3, diamond, test, outcomde);
}

 void test(...) {
  #pragma HLS INTERFACE s_axilite autorestart port=return

...
}

NOTE : This function must have the autorestart counter in the s_axilite adapter, as enabled via
#pragma HLS interface s_axilite port=return autorestart
```

Auto-Restart Limitations

The auto-restart limitations.

1. No infinite auto-restart: A finite number of restarts must be specified.
2. No support for direct I/O variables.
3. Input mutability during a single auto-restart session:
4. Pointer inputs: Test Bench cannot change pointed data between iterations.
5. Array inputs: DUT-originated updates are supported; test bench-originated updates are not.
6. Control protocol: Only ap_ctrl_hs is supported; ap_ctrl_chain and ap_ctrl_none are not supported.
7. Interfaces: Arrays to Stream FIFO are not supported.
8. No support for mailbox.

Using Auto-Restart in XRT

The following sections describe the auto-restarting HLS design examples.

Using Auto-Restart with the Mailbox

The mailbox feature provides the ability to have semi-synchronization with a software application. The mailbox is a non-blocking mechanism that updates the HLS design parameters. Any updates provided through the mailbox will be picked up the next time the design starts.
This example design uses scalar values which will be programmed from the software application, and the design will pick them at the next software call. The scalars, `adder1` and `adder2`, will be asynchronously updated from the software application.
Configure the HLS compiler for auto-restarting mode and enable the mailbox feature using the following configuration commands:

```
syn.interface.s_axilite_mailbox=both
syn.interface.s_axilite_auto_restart_counter=1
syn.interface.s_axilite_sw_reset=1
```

This informs the tool to create the IP or kernel using the autorestart mechanisms. The example HLS design code follows:

```
#define DWIDTH 32
11
12 typedef ap_axiu<DWIDTH, 0, 0, 0> pkt;
13
14 extern "C" {
15 void krnl_stream_vdatamover(hls::stream<pkt> &in,
16     ¦   ¦   ¦   ¦   ¦ hls::stream<pkt> &out,
17     ¦   ¦   ¦   ¦   ¦ int adder1,
18     ¦   ¦   ¦   ¦   ¦ int adder2
19     ¦   ¦   ¦   ¦   ¦ ) {
20
21 #pragma HLS interface ap_ctrl_chain port=return
22 #pragma HLS INTERFACE s_axilite port=adder2
25 #pragma  HLS port=adder1 stable
   #pragma  HLS port=adder2 stable
27 bool eos = false;
28 vdatamover:
29   do {
30     // Reading a and b streaming into packets
31     pkt t1 = in.read();
32
33     // Packet for output
```

```
34      pkt t_out;
35
36      // Reading data from input packet
37      ap_uint<DWIDTH> in1 = t1.data;
38
39      // Vadd operation
40      ap_uint<DWIDTH> tmpOut = in1+adder1+adder2;
41
42      // Setting data and configuration to output packet
43      t_out.data = tmpOut;
44      t_out.last = t1.last;
45      t_out.keep = -1; // Enabling all bytes
46
47      // Writing packet to output stream
48      out.write(t_out);
49
50      if (t1.last) {
51      ¦ eos = true;
52      }
53    } while (eos == false);
54
```

Create a mailbox to update the scalars values `adder1` and `adder2`.
Update the design parameters from the software application using the `set_arg` and `write` methods as shown below. The auto-restarting HLS design will not stop itself because there is no start and stop for a streaming interface. The module must be explicitly stopped or reset. The application code can explicitly stop the design from running using the `abort()` method.

```
// add(in1, in2, nullptr, data_size)
  xrt::kernel add(device, uuid, "krnl_stream_vadd");
  xrt::bo in1(device, data_size_bytes, add.group_id(0));
  auto in1_data = in1.map<int*>();
  xrt::bo in2(device, data_size_bytes, add.group_id(1));
  auto in2_data = in2.map<int*>();

  // mult(in3, nullptr, out, data_size)
  xrt::kernel mult(device, uuid, "krnl_stream_vmult");
  xrt::bo in3(device, data_size_bytes, mult.group_id(0));
  auto in3_data = in3.map<int*>();
  xrt::bo out(device, data_size_bytes, mult.group_id(2));
  auto out_data = out.map<int*>();


  xrt::kernel incr(device, uuid, "krnl_stream_vdatamover");
  int adder1 = 20;  // arbitrarily chosen to be different from 0
  int adder2 = 10;  // arbitrarily chosen to be different from 0

  // create run objects for re-use in loop
  xrt::run add_run(add);
  xrt::run mult_run(mult);
  std::cout <<"performing never-ending mode with infinite auto restart"<<std::endl;
  auto incr_run = incr(xrt::autostart{0}, nullptr, nullptr, adder1, adder2);

// create mailbox to programatically update the incr scalar adder
  xrt::mailbox incr_mbox(incr_run);
```

```
// computed expected result
std::vector<int> sw_out_data(data_size);

std::cout << " for loop started" <<std::endl;
bool error = false;   // indicates error in any of the iterations
for (unsigned int cnt = 0; cnt < iter; ++cnt) {


  // Create the test data and software result
  for(size_t i = 0; i < data_size; ++i) {
    in1_data[i] = static_cast<int>(i);
    in2_data[i] = 2 * static_cast<int>(i);
    in3_data[i] = static_cast<int>(i);
    out_data[i] = 0;
    sw_out_data[i] = (in1_data[i] + in2_data[i] + adder1 + adder2) * in3_data[i];
  }

  // sync test data to kernel
  in1.sync(XCL_BO_SYNC_BO_TO_DEVICE);
  in2.sync(XCL_BO_SYNC_BO_TO_DEVICE);
  in3.sync(XCL_BO_SYNC_BO_TO_DEVICE);

  // start the pipeline
  add_run(in1, in2, nullptr, data_size);
  mult_run(in3, nullptr, out, data_size);

  // wait for the pipeline to finish
  add_run.wait();
  mult_run.wait();

  // prepare for next iteration, update the mailbox with the next
  // value of 'adder'.
  incr_mbox.set_arg(2, ++adder1); // update the mailbox
  incr_mbox.set_arg(3, --adder2); // update the mailbox


  // write the mailbox content to hw, the write will not be picked
  // up until the next iteration of the pipeline (incr).
  incr_mbox.write();  // requests sync of mailbox to hw


  // sync result from device to host
  out.sync(XCL_BO_SYNC_BO_FROM_DEVICE);

  // compare with expected scalar adders
  for (size_t i = 0 ; i < data_size; i++) {
    if (out_data[i] != sw_out_data[i]) {
      std::cout << "error in iteration = " << cnt
                << " expected output = " << sw_out_data[i]
                << " observed output = " << out_data[i]
                << " adder1 = " << adder1 - 1
                << " adder2 = " << adder2 + 1 << '\n';
      throw std::runtime_error("result mismatch");
    }
  }
}
```

```
        incr_run.abort();
}
```

Using Counted Auto-Restart

This example uses the same code as the prior example, but in this case the HLS design is configured to run and restart for three iterations and then stop. The only required change is to specify the number of iterations from the software application as shown in the code example below.

★ **Tip:** In the counted auto-restart case, there is no need to use the `abort()`, because the software application will know when to stop.

```
143   xrt::kernel incr(device, uuid, "increment");
144   int adder1 = 20;  // arbitrarily chosen to be different from 0
145   int adder2 = 10;  // arbitrarily chosen to be different from 0
151   // start the incr kernel in auto restart mode with default adders
152   // since it is a streaming kernel it will be stalled waiting for
153   // input
154   auto incr_run = incr(xrt::autostart{3}, nullptr, nullptr, adder1);
```

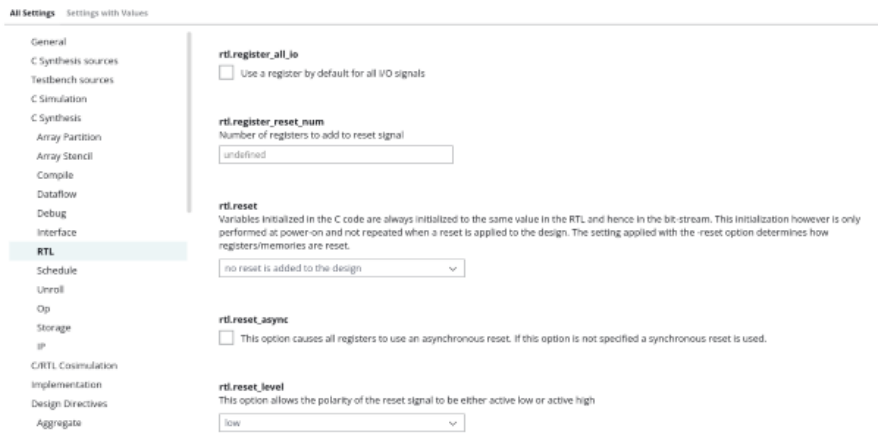
Controlling Initialization and Reset Behavior

The reset port is used in a device to return the registers and block RAM connected to the reset port to an initial value any time the reset signal is applied. Typically the most important aspect of RTL configuration is selecting the reset behavior.

Note: When discussing reset behavior it is important to understand the difference between initialization and reset. Refer to Initialization Behavior for more information.

The presence and behavior of the RTL reset port is controlled using configuration commands as described in RTL Configuration, as shown in the following figure.

Figure: RTL Configurations



The reset settings include the ability to set the polarity of the reset and whether the reset is synchronous or asynchronous but more importantly it controls, through the reset option, which registers are reset when the reset signal is applied.

Important: When AXI4 interfaces are used on a design the reset polarity is automatically changed to active-Low irrespective of the setting in the `config_rtl` configuration. This is required by the AXI4 standard.

The reset option has four settings:

none

No reset is added to the design.

control
: This is the default and ensures all control registers are reset. Control registers are those used in state machines and to generate I/O protocol signals. This setting ensures the design can immediately start its operation state.

state
: This option adds a reset to control registers (as in the control setting) plus any registers or memories derived from static and global variables in the C/C++ code. This setting ensures static and global variable initialized in the C/C++ code are reset to their initialized value after the reset is applied.

all
: This adds a reset to all registers and memories in the design.

Finer grain control over reset is provided through the RESET pragma or directive as described in syn.directive.reset. Static and global variables can have a reset added through the RESET directive. Variables can also be removed from those being reset by using the RESET directive's `off` option.

!! Important: It is important when using the reset `state` or `all` options to consider the effect on resetting arrays.

Initialization Behavior

In C/C++, variables defined with the static qualifier and those defined in the global scope are initialized to zero, by default. These variables can optionally be assigned a specific initial value. For these initialized variables, the value in the C/C++ code is assigned at compile time (at time zero) and never again. In both cases, the initial value is implemented in the RTL.

- During RTL simulation the variables are initialized with the same values as the C/C++ code.
- The variables are also initialized in the bitstream used to program the FPGA. When the device powers up, the variables will start in their initialized state.

In the RTL, although the variables start with the same initial value as the C/C++ code, there is no way to force the variable to return to this initial state. To restore the initial state, variables must be implemented with a reset signal.

!! Important: Top-level function arguments can be implemented in an AXI4-Lite interface. Because there is no way to provide an initial value in C/C++ for function arguments, these variable cannot be initialized in the RTL as doing so would create an RTL design with different functional behavior from the C/C++ code which would fail to verify during C/RTL co-simulation.